

Правообладатель: [ПРАВООБЛАДАТЕЛЬ: ФИО полностью или наименование
организации]

FiberFleet — система планирования, учёта и контроля сервисного обслуживания тягачей и прицепов

Версия 1.0

ДЕПОНИРУЕМЫЕ МАТЕРИАЛЫ (исходный текст программы для ЭВМ)

Автор: [АВТОР 1: Фамилия Имя Отчество]

Язык программирования: JavaScript (ECMAScript 2022, Node.js, React/JSX), SQL (SQLite)

Исходный текст представлен фрагментами: страницы 1-34 и 149-183 листинга из 183 (первые 34 и последние 35 страниц).

Всего листов в документе: 70.

[Город], 2026

```
// ===== Файл: server/domain/checklist.js =====
// Шаблон дефектной ведомости (по форме «Дефектная ведомость» заказчика) и база правил:
// дефект → работы из справочника → зона осмотра (куда залезть, что проверить) → связанные прове
рки.
//
// Виды пунктов:
//   pm          – норма / дефект («+/-» в бумажной ведомости)
//   yesno       – есть / нет дефектов (дефект = «есть»)
//   volt        – замер напряжения, дефект если вне диапазона [min, max]
//   mileage     – работа «по пробегу», статус считается по правилам ТО (ключ maint)
//   positions   – отметка позиций на схеме осей (тягач/прицеп × ось × сторона)
//   damage      – фиксация новых повреждений (выделить цветом), работ не создаёт
//
// Работы задаются ссылкой "Категория::Название" на справочник. Фильтры подбора:
//   veh: tractor|trailer, axle: front|rear, brake: disc|drum.
// Приоритет: 1 – критично (безопасность, выпуск запрещён), 2 – высокий, 3 – плановый, 4 – по со
гласованию.
// seq: 0 – мойка, 1 – осмотр/диагностика, 2 – ремонт, 3 – регулировки после ремонта. Внутри рем
онта по одному ТС
// задачи с меньшим seq выполняются раньше.

export const PRIORITIES = {
  1: { label: 'Критично', hint: 'Безопасность – выпуск ТС без устранения запрещён', days: 1 },
  2: { label: 'Высокий', hint: 'Устранить в текущий заезд', days: 3 },
  3: { label: 'Плановый', hint: 'Можно совместить с ближайшим ТО', days: 5 },
  4: { label: 'По согласованию', hint: 'Выполняется после согласования', days: 10 },
};

export const DAMAGE_ZONES = [
  'Кабина – перед', 'Кабина – лев.', 'Кабина – прав.', 'Бампер', 'Крыло лев.', 'Крыло прав.',
  'Обтекатель', 'Топливный бак', 'Зеркала', 'Прицеп – лев. борт', 'Прицеп – прав. борт',
  'Прицеп – зад/ворота', 'Каркас тента', 'Пластик/подножки',
];

const AXLE_INSPECTION = 'Тормозная система::Осмотр оси (суппорта, колодки, диски, амортизатора)';
const AIR_LEAK = 'Дополнительные работы::Поиск и устранение утечки воздуха';

export const SECTIONS = [
  {
    id: 'wash', title: 'Проверка качества мойки', vehicle: 'tractor',
    note: 'Без качественной мойки дефектовку узлов провести нельзя – повторная мойка ставится пе
рвой.',
    items: [
      { id: 'wash_reducer', label: 'Редуктор' },
      { id: 'wash_under_cab', label: 'Под кабиной' },
      { id: 'wash_tanks', label: 'Баки' },
      { id: 'wash_cabin', label: 'Чистота салона' },
    ].map((i) => ({
      ...i, kind: 'pm', priority: 3, seq: 0,
      works: [{ wt: 'Мойка::Мойка тягач' }],
    })),
  },
];
```

```

        zone: `Мойка: повторно промыть – ${i.label.toLowerCase()}`,
    })),
},
{
    id: 'damage', title: 'Контроль новых повреждений ТС',
    note: 'Фиксация повреждений при приёмке. Работы не создаются автоматически – при необходимости добавьте «Кузовной ремонт (по согласованию)».',
    items: [
        { id: 'dmg_body', label: 'Царапины, вмятины, сколы', kind: 'damage', priority: 4 },
        { id: 'dmg_plastic', label: 'Повреждения пластика, крыла, каркаса тента', kind: 'damage', priority: 4 },
    ],
},
{
    id: 'steering', title: 'Рулевое управление', vehicle: 'tractor',
    items: [
        {
            id: 'steer_cross', label: 'Тяга поперечная', kind: 'pm', priority: 1, seq: 2,
            works: [{ wt: 'Рулевое управление::С/у рулевой тяги (поперечной)' }],
            zone: 'Передняя ось, за балкой: поперечная рулевая тяга – люфт шарниров, пыльники, изгиб тяги',
            related: [{ wt: 'Рулевое управление::Диагностика рулевого управление', scope: 'vehicle' },
            why: 'Неисправность рулевой тяги – проверить весь рулевой привод' ]],
            checks: ['Рулевые наконечники', 'Шкворни (люфт)', 'Схождение колёс после замены'],
            options: ['Рулевое управление::С/у рулевого наконечника'],
        },
        {
            id: 'steer_long', label: 'Тяга продольная', kind: 'pm', priority: 1, seq: 2,
            works: [{ wt: 'Рулевое управление::С/у рулевой тяги (продольной)' }],
            zone: 'Левая сторона, под кабиной: сошка – продольная тяга – рычаг поворотного кулака; люфт шарниров',
            related: [{ wt: 'Рулевое управление::Диагностика рулевого управление', scope: 'vehicle' },
            why: 'Неисправность рулевой тяги – проверить весь рулевой привод' ]],
            checks: ['Крепление рулевой сошки', 'Люфт на выходном валу рулевого редуктора'],
            options: ['Рулевое управление::С/у рулевой сошки'],
        },
        {
            id: 'steer_gur', label: 'ГУР', kind: 'pm', priority: 1, seq: 1,
            works: [{ wt: 'Рулевое управление::Диагностика рулевого управление' }],
            zone: 'Бачок ГУР (уровень, пена), шланги высокого давления, рулевой редуктор (течи), насос ГУР (шум)',
            checks: ['Фильтр ГУР', 'Натяжение/состояние приводного ремня'],
            options: [
                'Рулевое управление::С/у насоса гидроусилителя',
                'Рулевое управление::С/у рулевого редуктора (ГУР)',
                'ДВС::Замена фильтра (топ/воз./влагоотд./пром. Центрифуги/ГУР/КПП)',
            ],
        },
    ],
},
},

```

```

{
  id: 'engine', title: 'Двигатель и Т0', vehicle: 'tractor',
  items: [
    {
      id: 'eng_to', label: 'Т0', kind: 'mileage', maint: 'to', priority: 2, seq: 2,
      zone: 'Моторное масло и фильтры (масляный, топливный, воздушный, сепаратор), смазка узло
в шасси',
      options: ['ДВС::Замена масла (1-2 фильтра)', 'Трансмиссия::Шприцевание (1 точка)'],
    },
    {
      id: 'eng_valves', label: 'Регулировка клапанов', kind: 'mileage', maint: 'valves', prior
ity: 3, seq: 2,
      zone: 'Головка блока: снять клапанную крышку, проверить зазоры по таблице производителя'
,
      options: ['ДВС::Замена прокладки клапанной крышки'],
    },
    {
      id: 'eng_oil_leak', label: 'Потечи масла', kind: 'pm', priority: 2, seq: 1,
      works: [{ wt: 'Дополнительные работы::Устранение течи масла (поиск, замена уплотнений)'
}],
      zone: 'ДВС: клапанная крышка, поддон, передний/задний сальник коленвала, теплообменник,
турбокомпрессор; КПП и редуктор – сальники',
      checks: ['Уровень масла ДВС', 'Картерные газы (сапун)', 'Масло во впуске (турбина)'],
      options: ['ДВС::Замена прокладки клапанной крышки', 'ДВС::Замена прокладки поддона', 'ДВ
С::Замена масляного радиатора (теплообменник)'],
    },
    {
      id: 'eng_fuel_leak', label: 'Потечи ДТ', kind: 'pm', priority: 1, seq: 2,
      works: [{ wt: 'ДВС::Найти, устранить утечку топлива/подсос воздуха в топливе' }],
      zone: 'Топливные баки и пробки, магистрали и штуцеры, ТННД/ТНВД, фильтры грубой/тонкой о
чистки, обратка форсунок',
      checks: ['Подсос воздуха (прозрачная трубка до ТННД)', 'Ручная подкачка'],
      options: ['ДВС::Снять/поставить топливный бак', 'ДВС::С/у ТНВД с/у ТННД', 'ДВС::Замена ф
ильтра (топ/воз./влагоотд./пром. Центрифуги/ГУР/КПП)'],
    },
  ],
},
{
  id: 'brakes', title: 'Тормозная система а/м',
  items: [
    {
      id: 'br_receivers', label: 'Ресиверы, наличие конденсата', kind: 'pm', vehicle: 'tractor
', priority: 2, seq: 1,
      works: [{ wt: 'Дополнительные работы::Слив конденсата, проверка осушителя воздуха' }],
      zone: 'Слить конденсат из всех ресиверов; масло в конденсате – износ компрессора; картри
дж осушителя воздуха',
      checks: ['Компрессор (выброс масла)', 'Регулятор давления'],
      options: ['ДВС::Замена фильтра (топ/воз./влагоотд./пром. Центрифуги/ГУР/КПП)', 'ДВС::С/у
компрессор'],
    },
  ],
}

```

```

{
  id: 'br_leak_tractor', label: 'Утечка воздуха тягача', kind: 'pm', vehicle: 'tractor', priority: 1, seq: 2,
  works: [{ wt: AIR_LEAK }],
  zone: 'Магистралы и фитинги, ресиверы, кран стояночного тормоза, тормозные камеры и энергоаккумуляторы, пневмоподвеска',
  related: [{ wt: 'Тормозная система:Диагностика тормозов', scope: 'vehicle', why: 'Утечка воздуха – проверить тормозную систему целиком' }],
  relatedItems: ['br_receivers', 'br_spring'],
},
{
  id: 'br_leak_trailer', label: 'Утечка воздуха п/п', kind: 'pm', vehicle: 'trailer', priority: 1, seq: 2,
  works: [{ wt: AIR_LEAK }],
  zone: 'Соединительные головки и шланги («пальмы»), ресиверы прицепа, ускорительный клапан, тормозные камеры, пневмоподушки',
  related: [{ wt: 'Прицеп:Диагностика тормозов прицепа', scope: 'vehicle', why: 'Утечка воздуха – проверить тормоза прицепа' }],
},
{
  id: 'br_leak_pedal', label: 'Утечка воздуха при нажатии на тормоз', kind: 'pm', vehicle: 'tractor', priority: 1, seq: 2,
  works: [{ wt: AIR_LEAK }],
  zone: 'Главный тормозной кран, ускорительные клапаны, мембраны тормозных камер, модуляторы EBS – при нажатой педали',
  related: [{ wt: 'Тормозная система:Диагностика тормозов', scope: 'vehicle', why: 'Утечка при торможении – проверить тормозную систему целиком' }],
  options: ['Тормозная система:С/у главного тормозного крана', 'Тормозная система:С/у тормозной камеры'],
},
{
  id: 'br_pads', label: 'Колодки', kind: 'positions', priority: 1, seq: 2,
  works: [
    { veh: 'tractor', brake: 'disc', axle: 'front', wt: 'Тормозная система:С/у колодок (диск) передние' },
    { veh: 'tractor', brake: 'disc', axle: 'rear', wt: 'Тормозная система:С/у колодок (диск) задние' },
    { veh: 'tractor', brake: 'drum', axle: 'front', wt: 'Тормозная система:С/у колодок (барабан) перед' },
    { veh: 'tractor', brake: 'drum', axle: 'rear', wt: 'Тормозная система:С/у колодок (барабан)зад' },
    { veh: 'trailer', brake: 'disc', wt: 'Тормозная система:С/у колодок (диск) задние' },
    { veh: 'trailer', brake: 'drum', wt: 'Прицеп:С/у тормозных колодок (барабан)' },
  ],
  zone: '{pos}: снять колесо, замерить остаток накладок, осмотреть диск/барабан (трещины, износ), направляющие суппорта',
  related: [{ wt: AXLE_INSPECTION, scope: 'axle', why: 'Колодки меняются комплектом на ось – осмотреть всю ось' }],
  relatedItems: ['br_discs', 'br_calipers'],
},

```

```

{
  id: 'br_calipers', label: 'Суппорта', kind: 'positions', priority: 1, seq: 2,
  works: [
    { veh: 'tractor', axle: 'front', wt: 'Тормозная система::С/у суппорта перед' },
    { veh: 'tractor', axle: 'rear', wt: 'Тормозная система::С/у суппорта зад' },
    { veh: 'trailer', wt: 'Тормозная система::С/у суппорта зад' },
  ],
  zone: '{pos}: снять колесо; ход суппорта на направляющих, пыльники, механизм подвода, ра-
вномерность износа колодок',
  related: [{ wt: AXLE_INSPECTION, scope: 'axle', why: 'Неисправность суппорта – проверка
оси (колодки, диски, ступичный подшипник, амортизатор)' }],
  checks: ['Ступичный подшипник (люфт) на этой оси', 'Перегрев диска (цвета побежалости)'],
  relatedItems: ['br_pads', 'br_discs'],
  options: [
    'Тормозная система::Полная переборка суппорта (внутренний механизм+направляющие, со сн-
ятием суппортов)',
    'Тормозная система::Замена РМК (смазка, переборка внутреннего механизма без снятия суп-
порта)',
  ],
},
{
  id: 'br_discs', label: 'Тормозные диски / барабаны', kind: 'positions', priority: 1, seq:
: 2,
  works: [
    { veh: 'tractor', brake: 'disc', axle: 'front', wt: 'Тормозная система::С/у тормозного
диска(перед)' },
    { veh: 'tractor', brake: 'disc', axle: 'rear', wt: 'Тормозная система::С/у тормозного
диска(зад)' },
    { veh: 'tractor', brake: 'drum', axle: 'front', wt: 'Тормозная система::С/у барабана (
перед)' },
    { veh: 'tractor', brake: 'drum', axle: 'rear', wt: 'Тормозная система::С/у барабана (з
ад)' },
    { veh: 'trailer', brake: 'disc', wt: 'Тормозная система::С/у тормозного диска(зад)' },
    { veh: 'trailer', brake: 'drum', wt: 'Прицеп::С/у тормозного барабана' },
  ],
  zone: '{pos}: толщина диска/диаметр барабана, трещины, задиры, биение',
  related: [{ wt: AXLE_INSPECTION, scope: 'axle', why: 'Износ диска/барабана – проверка ос-
и и колодок' }],
  checks: ['При замене диска колодки на этой оси менять обязательно'],
  relatedItems: ['br_pads', 'br_calipers'],
  options: ['Прицеп::Проточка тормозного барабана'],
},
{
  id: 'br_spring', label: 'Энергоаккумулятор', kind: 'positions', priority: 1, seq: 2,
  works: [{ wt: 'Тормозная система::С/у энергоаккумулятора' }],
  zone: '{pos}: корпус и мембрана (утечка), шток, вилка, регулятор (трещотка), шланги',
  checks: ['Ход штока при торможении', 'Работа трещотки (авторегулятора)'],
  options: ['Тормозная система::С/у тормозной камеры', 'Тормозная система::С/у тормозного
рычага'],
}

```

```

        relatedItems: ['br_leak_tractor'],
    },
],
},
{
    id: 'cooling', title: 'Система охлаждения', vehicle: 'tractor',
    items: [
        {
            id: 'cool_tensioners', label: 'Подшипники натяжителей', kind: 'pm', priority: 2, seq: 2,
            works: [{ wt: 'ДВС::С/у натяжителя ремня' }],
            zone: 'Привод навесных агрегатов: натяжитель и обводные ролики (шум, люфт), ремень (трещины, расслоение)',
            checks: ['Помпа (люфт, течь из дренажа)', 'Вискомуфта', 'Шкив генератора'],
            relatedItems: ['el_generator'],
            options: ['ДВС::Замена/смазка подшипников, роликов', 'ДВС::Замена ремня', 'ДВС::Помпа'],
        },
        {
            id: 'cool_leak', label: 'Потеки антифриза', kind: 'pm', priority: 2, seq: 2,
            works: [{ wt: 'Дополнительные работы::Устранение течи антифриза (патрубки, хомуты)' }],
            zone: 'Радиатор, патрубки и хомуты, помпа (дренажное отверстие), расширительный бачок, отопитель, теплообменник',
            checks: ['Эмульсия в масле (антифриз в масле → прокладка ГБЦ/теплообменник)', 'Уровень ОЖ'],
            options: ['ДВС::Помпа', 'ДВС::С/у радиатора +интеркуллер', 'ДВС::Замена масляного радиатора (теплообменник)'],
        },
    ],
},
{
    id: 'electric', title: 'Электросистема', vehicle: 'tractor',
    items: [
        {
            id: 'el_generator', label: 'Генератор', kind: 'volt', unit: 'В', min: 27.5, max: 29.0, priority: 2, seq: 1,
            hint: 'Заведённый двигатель, 1500 об/мин. Норма 27,5–29,0 В',
            works: [{ wt: 'Дополнительные работы::Диагностика электросистемы (зарядка, проводка)' }],
            zone: 'Генератор: напряжение на клеммах, ремень привода, силовые клеммы и «масса»',
            checks: ['Ниже нормы – недозаряд (щётки, диодный мост)', 'Выше нормы – регулятор напряжения, риск выкипания АКБ'],
            relatedItems: ['el_battery'],
            options: ['ДВС::С/у генератор', 'ДВС::Замена ремня'],
        },
        {
            id: 'el_lights', label: 'Фонари/фары/дефекты', kind: 'yesno', priority: 2, seq: 2,
            works: [{ wt: 'Дополнительные работы::Замена ламп/фонарей/фар' }],
            zone: 'Фары, габариты, стоп-сигналы, поворотники тягача и прицепа; розетка/вилка 15-pin, жгут прицепа',
        },
    ],
}

```

```

        id: 'el_battery', label: 'Аккумулятор (не заведённое авто)', kind: 'volt', unit: 'В', min: 24.6, max: 26.0, priority: 2, seq: 1,
        hint: 'Двигатель заглушен, масса включена ≥ 1 ч. Норма 24,6–26,0 В',
        works: [{ wt: 'Дополнительные работы::Проверка и зарядка АКБ' }],
        zone: 'Отсек АКБ: напряжение без нагрузки, клеммы (окислы, затяжка), крепление, ток утечки',
        relatedItems: ['el_generator'],
        options: ['Дополнительные работы::С/у АКБ'],
    },
],
},
{
    id: 'driveline', title: 'КПП / Карданы', vehicle: 'tractor',
    items: [
        {
            id: 'dl_crosses', label: 'Крестовины', kind: 'pm', priority: 2, seq: 2,
            works: [{ wt: 'Трансмиссия::Замена креставины карданного вала' }],
            zone: 'Карданные валы: люфт крестовин (покачать вилку), шлицы, подвесной подшипник, болты фланцев',
            checks: ['Сальник хвостовика редуктора', 'Сальник выходного вала КПП'],
            options: ['Трансмиссия::С/у карданного вала', 'Трансмиссия::С/у подвесного подшипника', 'Трансмиссия::Шприцевание (1 точка)'],
        },
    ],
},
{
    id: 'tires', title: 'Состояние шин',
    items: [
        {
            id: 'tires', label: 'Шины (отметьте колёса с дефектом)', kind: 'positions', priority: 2, seq: 2,
            works: [{ wt: 'Дополнительные работы::Шиномонтаж (с/у, замена шины 1 колесо)' }],
            zone: '{pos}: остаток протектора, порезы, грыжи, давление, характер износа',
            checks: ['Неравномерный износ передних шин – сходжение/рулевое', 'Неравномерный износ шин прицепа – соосность балок'],
            options: ['Ходовая часть прицепа::Выставление соосности балки', 'Рулевое управление::Диагностика рулевого управления'],
        },
    ],
},
{
    id: 'suspension', title: 'Подвеска',
    items: [
        {
            id: 'sus_stab', label: 'Стабилизаторы', kind: 'pm', vehicle: 'tractor', priority: 2, seq: 2,
            works: [{ wt: 'Подвеска::Замена втулок стабилизатора (перед)' }],
            zone: 'Стабилизаторы перед/зад: втулки, стойки (шарниры), крепление к раме',
            related: [{ wt: 'Подвеска::Диагностика подвески', scope: 'vehicle', why: 'Дефект стабилизатора – диагностика подвески' }],
        },
    ],
},

```

```

    options: ['Подвеска::С/у стойки стабилизатора(перед)', 'Подвеска::С/у стойки стабилизато
ра (зад)'],
  },
  {
    id: 'sus_cab', label: 'Сайлентблоки кабины', kind: 'pm', vehicle: 'tractor', priority: 3
, seq: 2,
    works: [{ wt: 'Подвеска кабины::Замена сайлентблока/втулки/цилиндра/подъема кабины' }],
    zone: 'Передние опоры кабины (торсион), задние опоры: амортизаторы и пневмоподушки кабин
ы, замок',
    options: ['Подвеска кабины::С/у амортизатора кабин', 'Подвеска кабины::С/у пневмоподушки
кабин'],
  },
  {
    id: 'sus_springs', label: 'Сайлентблоки рессор', kind: 'pm', vehicle: 'tractor', priorit
y: 2, seq: 2,
    works: [{ wt: 'Подвеска::Замена сайлентблока полурессоры' }],
    zone: 'Передние рессоры: серьги, втулки ушек, пальцы; задние полурессоры: сайлентблоки,
стремянки',
    related: [{ wt: 'Подвеска::Диагностика подвески', scope: 'vehicle', why: 'Износ сайлентб
локов – диагностика подвески' }],
    options: ['Подвеска::Замена втулки рессоры (рессора снята)', 'Подвеска::С/у рессоры (пер
ед)'],
  },
  {
    id: 'sus_air', label: 'Пневморессоры', kind: 'positions', priority: 2, seq: 2,
    works: [{ wt: 'Прицеп::С/у пневмоподушки' }],
    zone: '{pos}: пневмоподушка – трещины, потёртости, утечка (мыльный раствор), поршень, кр
епление',
    checks: ['Кран уровня пола и тяга датчика'],
    options: ['Автомобиль::Калибровка уровня пола'],
  },
  {
    id: 'sus_silent', label: 'Сайлентблоки', kind: 'positions', priority: 2, seq: 2,
    works: [
      { veh: 'tractor', wt: 'Подвеска::Замена сайлентблока реактивной тяги/стойки стабилизат
ора' },
      { veh: 'trailer', wt: 'Ходовая часть прицепа::Замена сайлентблока в коробах (одного)'
},
    ],
    zone: '{pos}: сайлентблоки реактивных тяг / в коробах подвески – разрывы резины, смещени
е втулки',
    related: [
      { veh: 'trailer', wt: 'Ходовая часть прицепа::Выставление соосности балки', scope: 'ax
le', seq: 3, why: 'После замены сайлентблоков в коробах – выставить соосность балки' },
      { veh: 'tractor', wt: 'Подвеска::Диагностика подвески', scope: 'vehicle', why: 'Износ
сайлентблоков – диагностика подвески' },
    ],
  },
  {
    id: 'sus_shocks', label: 'Амортизаторы', kind: 'positions', priority: 2, seq: 2,

```

```

    works: [
      { veh: 'tractor', wt: 'Подвеска::С/у амортизатора' },
      { veh: 'trailer', wt: 'Прицеп::С/у амортизатора' },
    ],
    zone: '{pos}: течь масла, отбойники, верхний/нижний кронштейн и сайлентблоки',
    related: [{ wt: AXLE_INSPECTION, scope: 'axle', why: 'Дефект амортизатора – осмотр оси'
  }],
  options: ['Прицеп::С/у кронштейна амортизатора'],
},
{
  id: 'sus_other', label: 'Подвеска – прочее', kind: 'pm', vehicle: 'tractor', priority: 2
, seq: 1,
  works: [{ wt: 'Подвеска::Диагностика подвески' }],
  zone: 'Укажите узел в комментарии',
},
],
},
{
  id: 'trailer', title: 'Прицеп – повреждения или требуется ремонт', vehicle: 'trailer',
  items: [
    {
      id: 'tr_frame', label: 'Рама прицепа', kind: 'pm', priority: 1, seq: 2,
      works: [{ wt: 'Дополнительные работы::Сварочные работы (рама/платформа/борта) за 1 н/ч'
    }],
      zone: 'Лонжероны и поперечины (трещины у сварных швов и кронштейнов подвески), шкворень
и плита сцепки',
      related: [{ wt: 'Прицеп::Диагностика ходовой части прицепа', scope: 'vehicle', why: 'Пов
реждение рамы – проверить кронштейны и ходовую' }],
      checks: ['Износ шкворня', 'Опорное устройство'],
    },
    {
      id: 'tr_platform', label: 'Платформа', kind: 'pm', priority: 3, seq: 2,
      works: [{ wt: 'Дополнительные работы::Сварочные работы (рама/платформа/борта) за 1 н/ч'
    }],
      zone: 'Настил пола, боковые балки, точки крепления груза',
    },
    {
      id: 'tr_sides', label: 'Борта', kind: 'pm', priority: 3, seq: 2,
      works: [{ wt: 'Дополнительные работы::Сварочные работы (рама/платформа/борта) за 1 н/ч'
    }],
      zone: 'Борта, запоры, петли',
    },
    {
      id: 'tr_slider', label: 'Сдвижное устройство', kind: 'pm', priority: 3, seq: 2,
      works: [{ wt: 'Дополнительные работы::Ремонт/регулировка сдвижного устройства' }],
      zone: 'Сдвижная крыша/рамка: ролики, направляющие, натяжители',
    },
    {
      id: 'tr_post', label: 'Стойка', kind: 'pm', priority: 3, seq: 2,
      works: [{ wt: 'Дополнительные работы::Ремонт стойки' }],

```

```

        zone: 'Стойки тента, ламели, крепление стоек',
    },
    {
        id: 'tr_cable', label: 'Таможенный трос', kind: 'pm', priority: 2, seq: 2,
        works: [{ wt: 'Дополнительные работы::Замена таможенного троса/ленты' }],
        zone: 'Трос TIR по периметру, наконечники, проушины',
    },
    {
        id: 'tr_tent', label: 'Тент', kind: 'pm', priority: 3, seq: 2,
        works: [{ wt: 'Дополнительные работы::Ремонт тента' }],
        zone: 'Тент: порывы, люверсы, ремни натяжения',
    },
    {
        id: 'tr_tape', label: 'Таможенная лента', kind: 'pm', priority: 2, seq: 2,
        works: [{ wt: 'Дополнительные работы::Замена таможенного троса/ленты' }],
        zone: 'Таможенная лента, крепления, пломбировочные устройства',
    },
    ],
},
];

// Плоский индекс пунктов
export const ITEMS = Object.fromEntries(
    SECTIONS.flatMap((s) => s.items.map((i) => [i.id, { ...i, section: s.id, vehicle: i.vehicle ||
        s.vehicle || 'any' }]])),
);

// Позиции на схеме осей. code: "<veh>:<axle>:<side>", side: L|R
export function axlePositions(vehType, axles) {
    const out = [];
    for (let a = 1; a <= axles; a++) {
        for (const side of ['L', 'R']) out.push({ code: `${vehType}:${a}:${side}`, vehType, axle: a,
            side });
    }
    return out;
}

export function positionLabel(code) {
    const [veh, axle, side] = String(code).split(':');
    const v = veh === 'tractor' ? 'Тягач' : 'Прицеп';
    const s = side === 'L' ? 'лев.' : side === 'R' ? 'прав.' : '';
    return `${v}, ось ${axle}${s ? ', ' + s : ''}`;
}

export function parsePosition(code) {
    const [veh, axle, side] = String(code).split(':');
    return { veh, axle: Number(axle), side };
}

// Все ссылки на справочник работ, используемые правилами (для проверки целостности)

```

```

export function referencedWorkKeys() {
  const keys = new Set();
  for (const item of Object.values(ITEMS)) {
    for (const w of item.works || []) keys.add(w.wt);
    for (const r of item.related || []) keys.add(r.wt);
    for (const o of item.options || []) keys.add(o);
  }
  return [...keys];
}

// Сериализуемый шаблон для клиента
export function publicTemplate() {
  return {
    priorities: PRIORITIES,
    damageZones: DAMAGE_ZONES,
    sections: SECTIONS.map((s) => ({
      id: s.id, title: s.title, note: s.note || null, vehicle: s.vehicle || 'any',
      items: s.items.map((i) => ({
        id: i.id, label: i.label, kind: i.kind, priority: i.priority,
        vehicle: i.vehicle || s.vehicle || 'any', unit: i.unit, min: i.min, max: i.max, hint: i.
hint,
        maint: i.maint, zone: i.zone, checks: i.checks || [], relatedItems: i.relatedItems || []
      ,
        options: i.options || [],
      })),
    })),
  });
}

// ===== Файл: server/domain/planner.js =====
// Формирование плана ремонта из дефектовки: работы, зоны осмотра, связанные проверки,
// приоритеты и сроки; проверка плана на противоречия.
import { ITEMS, PRIORITIES, SECTIONS, parsePosition, positionLabel } from './checklist.js';
import { vehicleMaintenance } from './maintenance.js';
import { fmtDateTime } from '../util.js';

export function workTypeIndex(db) {
  const rows = db.prepare('SELECT * FROM work_types WHERE active = 1').all();
  const byKey = new Map(rows.map((r) => [`${r.category}::${r.name}`, r]));
  const byId = new Map(rows.map((r) => [r.id, r]));
  return { byKey, byId };
}

const axleLabel = (veh, axle) => `${veh} === 'tractor' ? 'Тягач' : 'Прицеп'`, ось ${axle}`;

function isDefect(item, ans, maintByKey) {
  if (!ans) ans = {};
  switch (item.kind) {
    case 'pm':

```

```

    case 'yesno':
      return ans.state === 'defect';
    case 'volt': {
      if (ans.state === 'defect') return true;
      const v = ans.value === '' || ans.value == null ? null : Number(ans.value);
      return v != null && !Number.isNaN(v) && (v < item.min || v > item.max);
    }
    case 'mileage': {
      if (ans.state === 'defect') return true;
      if (ans.state === 'ok') return false;
      const st = maintByKey?.[item.maint];
      return st?.status === 'overdue';
    }
    case 'positions':
      return Array.isArray(ans.positions) && ans.positions.length > 0;
    default:
      return false;
  }
}

function matchWorks(item, veh, vehicleRec, axle) {
  return (item.works || []).filter((w) => {
    if (w.veh && w.veh !== veh) return false;
    if (w.brake && w.brake !== (vehicleRec?.brake_type || 'disc')) return false;
    if (w.axle) {
      const isFront = veh === 'tractor' && axle === 1;
      if ((w.axle === 'front') !== isFront) return false;
    }
    return true;
  });
}

/**
 * @param ctx { tractor, trailer, defects, release_deadline, inspected_at, tractor_mileage, trailer_mileage, cal }
 * @returns { tasks, inspection, damages, recommendations, warnings }
 */
export function generatePlan(db, ctx) {
  const { byKey, byId } = workTypeIndex(db);
  const { tractor, trailer, cal } = ctx;
  const defects = ctx.defects || {};
  const answers = defects.items || {};
  const inspectedAt = ctx.inspected_at || new Date().toISOString();
  const vehicles = { tractor, trailer };

  const maint = {};
  if (tractor) {
    maint.tractor = Object.fromEntries(vehicleMaintenance(db, 'tractor', { ...tractor, mileage: Math.max(tractor.mileage || 0, ctx.tractor_mileage || 0) }).map((s) => [s.key, s]));
  }
}

```

```

if (trailer) {
  maint.trailer = Object.fromEntries(vehicleMaintenance(db, 'trailer', { ...trailer, mileage:
Math.max(trailer.mileage || 0, ctx.trailer_mileage || 0) }).map((s) => [s.key, s]));
}

const tasks = new Map(); // key → task
const warnings = [];
const inspection = [];
const damages = [];
const recommendations = [];

const deadlineFor = (priority) => ctx.release_deadline || cal.addWorkDays(inspectedAt, PRIORIT
IES[priority]?.days ?? 5);

const addTask = ({ wt, veh, location, reason, source, seq, priority }) => {
  const vrec = vehicles[veh];
  if (!vrec) return null;
  const key = `${wt.id}|${veh}|${location || ''}`;
  const prev = tasks.get(key);
  if (prev) {
    if (!prev.reasons.includes(reason)) prev.reasons.push(reason);
    if (priority < prev.priority) { prev.priority = priority; prev.deadline = deadlineFor(prio
rity); }
    if (source === 'defect' && prev.source !== 'defect') { prev.source = 'defect'; prev.seq =
seq; }
    return prev;
  }
  const t = {
    key, work_type_id: wt.id, category: wt.category, description: wt.name,
    vehicle_type: veh, vehicle_id: vrec.id, location: location || null,
    reasons: [reason], source, seq, priority,
    deadline: deadlineFor(priority), hours: wt.hours, price: wt.price, salary: wt.salary,
    mileage: veh === 'tractor' ? ctx.tractor_mileage ?? tractor?.mileage : ctx.trailer_mileage
?? trailer?.mileage,
  };
  tasks.set(key, t);
  return t;
};

const resolveWt = (key, itemLabel) => {
  const wt = byKey.get(key);
  if (!wt) warnings.push(`Работа «${key.split(':')[1]}» (пункт «${itemLabel}») не найдена в с
правочнике – обновите справочник работ`);
  return wt;
};

// Сначала – дефекты, затем связанные проверки (чтобы не дублировать уже назначенные работы)
const relatedQueue = [];

for (const section of SECTIONS) {

```

```

for (const raw of section.items) {
  const item = ITEMS[raw.id];
  const ans = answers[item.id] || {};

  if (item.kind === 'damage') {
    if (ans.state === 'defect' || (ans.zones && ans.zones.length)) {
      damages.push({ item: item.id, label: item.label, zones: ans.zones || [], comment: ans.
comment || '' });
    }
    continue;
  }

  if (!isDefect(item, ans, maint[item.vehicle] === 'trailer' ? 'trailer' : 'tractor')) conti
nue;

  const comment = ans.comment ? ` - ${ans.comment}` : '';
  const reason = `Дефектовка: ${item.label}${comment}`;

  // Сгруппировать по ТС и оси
  let targets;
  if (item.kind === 'positions') {
    targets = ans.positions.map((code) => {
      const p = parsePosition(code);
      return { veh: p.veh, axle: p.axle, location: positionLabel(code), code };
    });
  } else {
    const veh = item.vehicle === 'any' ? 'tractor' : item.vehicle;
    targets = [{ veh, axle: null, location: null }];
  }

  const zoneEntry = {
    item: item.id, label: item.label, section: section.title, priority: item.priority,
    positions: [], vehicles: new Set(), checks: item.checks || [], zone: item.zone || '',
    value: item.kind === 'volt' ? ans.value : undefined, unit: item.unit, comment: ans.comme
nt || '',
  };

  for (const t of targets) {
    if (!vehicles[t.veh]) {
      warnings.push(`Пункт «${item.label}»: ${t.veh === 'trailer' ? 'прицеп' : 'тягач'} не у
казан в дефектовке – работа не запланирована`);
      continue;
    }
    zoneEntry.vehicles.add(t.veh);
    if (t.location) zoneEntry.positions.push(t.location);

    let works;
    if (item.kind === 'mileage') {
      const st = maint[t.veh]?.[item.maint];
      const wt = st?.work_type_id ? byId.get(st.work_type_id) : null;
      if (!wt) { warnings.push(`Для «${item.label}» не задано правило ТО для этой марки – до

```

```

        бавьте работу вручную`); continue; }
        works = [wt];
    } else {
        works = matchWorks(item, t.veh, vehicles[t.veh], t.axle).map((w) => resolveWt(w.wt, item.label)).filter(Boolean);
        if (!works.length) {
            warnings.push(`Пункт «${item.label}» (${t.location} || t.veh): нет подходящей работы в справочнике – добавьте вручную`);
        }
    }
    for (const wt of works) {
        addTask({ wt, veh: t.veh, location: t.location, reason, source: 'defect', seq: item.seq ?? 2, priority: item.priority });
    }
    // Дополнительные работы, отмеченные мастером в пункте
    for (const optKey of ans.options || []) {
        const wt = resolveWt(optKey, item.label);
        if (wt) addTask({ wt, veh: t.veh, location: t.location, reason: `${reason} (доп. работа)`, source: 'defect', seq: item.seq ?? 2, priority: item.priority });
    }
    for (const r of item.related || []) {
        if (r.veh && r.veh !== t.veh) continue;
        relatedQueue.push({ r, t, item });
    }
}

zoneEntry.vehicles = [...zoneEntry.vehicles];
zoneEntry.zone_text = item.zone
    ? item.zone.replace('{pos}', zoneEntry.positions.length ? zoneEntry.positions.join('; ') : (zoneEntry.vehicles[0] === 'trailer' ? 'Прицеп' : 'Тягач'))
    : '';
zoneEntry.related_labels = (item.relatedItems || []).map((id) => ITEMS[id]?.label).filter(Boolean);
inspection.push(zoneEntry);

// Рекомендация проверить связанные пункты, которые ещё не осмотрены
for (const relId of item.relatedItems || []) {
    const rel = ITEMS[relId];
    const relAns = answers[relId];
    const checked = relAns && (relAns.state || (relAns.positions && relAns.positions.length) || (relAns.value !== null && relAns.value !== ''));
    if (rel && !checked) {
        recommendations.push({ item: relId, label: rel.label, because: item.label, text: `Проверьте также «${rel.label}» – связано с дефектом «${item.label}»` });
    }
}
}
}

for (const { r, t, item } of relatedQueue) {

```

```

    const wt = resolveWt(r.wt, item.label);
    if (!wt) continue;
    const location = r.scope === 'axle' && t.axle ? axleLabel(t.veh, t.axle) : null;
    // если такая же работа уже назначена как ремонт (на ось или позицию этой оси) – не дублируе
М
    const dup = [...tasks.values()].some((x) => x.work_type_id === wt.id && x.vehicle_type === t
.veh && x.source === 'defect'
        && (!location || !x.location || x.location.startsWith(location)));
    if (dup) continue;
    addTask({ wt, veh: t.veh, location, reason: `Связанная проверка: ${r.why}`, source: 'related
', seq: r.seq ?? 1, priority: item.priority });
}

// T0 «на подходе», не отмеченное в ведомости – рекомендация
for (const veh of ['tractor', 'trailer']) {
    for (const st of Object.values(maint[veh] || {})) {
        const item = Object.values(ITEMS).find((i) => i.kind === 'mileage' && i.maint === st.key);
        const already = [...tasks.values()].some((x) => x.work_type_id === st.work_type_id && x.ve
hicle_type === veh);
        if (already) continue;
        if (st.status === 'overdue' || st.status === 'soon') {
            const wt = byId.get(st.work_type_id);
            recommendations.push({
                item: item?.id || null, maint: st.key, vehicle_type: veh, work_type_id: st.work_type_i
d,
                label: st.name, text: `${veh === 'tractor' ? 'Тягач' : 'Прицеп'}: ${st.name} – ${st.st
atus === 'overdue' ? 'просрочено' : 'скоро'}`
                + (st.remaining_km !== null ? ` (осталось ${st.remaining_km.toLocaleString('ru-RU')}
км)` : '')
                + (wt ? ` . Рекомендуется добавить «${wt.name}»` : ''),
            });
        }
    }
}

// Работы, добавленные вручную из справочника
for (const ex of defects.extra || []) {
    const wt = byId.get(Number(ex.work_type_id));
    if (!wt) continue;
    const veh = ex.vehicle_type === 'trailer' ? 'trailer' : 'tractor';
    const pr = Number(ex.priority) || 2;
    addTask({ wt, veh, location: ex.location || null, reason: `Добавлено вручную${ex.comment ? '
': ' + ex.comment : ''}`, source: 'manual', seq: wt.name.toLowerCase().startsWith('диагност') ?
1 : 2, priority: pr });
}

const list = [...tasks.values()].map((t) => ({ ...t, reason: t.reasons.join('; ') }));
list.forEach((t) => delete t.reasons);
list.sort((a, b) => a.seq - b.seq || a.priority - b.priority || a.vehicle_type.localeCompare(b
.vehicle_type));

```

```

    return { tasks: list, inspection, damages, recommendations, warnings: [...new Set(warnings)] }
;
}

```

```
const fmt = (iso) => fmtDateTime(iso, 180);
```

```
/**
```

```

 * Проверка плана: противоречия приоритетов/сроков, сроки в прошлом, позже даты выпуска,
 * нет допущенного исполнителя, расписание не укладывается в срок.
 */

```

```

export function validatePlan(order, tasks, { scheduleByTask = new Map(), unschedulable = new Map
(), now = new Date(), fmtDate = fmt } = {}) {
  const errors = [];
  const active = tasks.filter((t) => t.status !== 'done' && t.status !== 'cancelled');
  const nowIso = now.toISOString();

```

```

  for (const t of active) {
    if (t.deadline && t.deadline < nowIso && t.status === 'planned') {
      errors.push({ code: 'deadline_past', task_id: t.id, message: `«${t.description}»: срок ${f
mtDate(t.deadline)} уже прошёл` });
    }
    if (order.release_deadline && t.deadline && t.deadline > order.release_deadline) {
      errors.push({ code: 'after_release', task_id: t.id, message: `«${t.description}»: срок ${f
mtDate(t.deadline)} позже даты выпуска ТС ${fmtDate(order.release_deadline)}` });
    }
  }
}

```

```
// Противоречие приоритетности и сроков: более важная работа со сроком позже менее важной
```

```
const conflicts = new Set();
```

```

for (const a of active) {
  for (const b of active) {
    if (a.id === b.id || !a.deadline || !b.deadline) continue;
    if (a.priority < b.priority && a.deadline > b.deadline) {
      const k = `${a.id}-${b.id}`;
      if (conflicts.has(k)) continue;
      conflicts.add(k);
      errors.push({
        code: 'priority_conflict', task_id: a.id,
        message: `Противоречие приоритетов и сроков: «${a.description}» (приоритет ${a.priorit
y}) имеет срок ${fmtDate(a.deadline)} – позже, чем менее важная «${b.description}» (приоритет
${b.priority}, срок ${fmtDate(b.deadline)})`,
      });
    }
  }
}
}

```

```

for (const t of active) {
  const why = unschedulable.get(t.id);
  if (why) { errors.push({ code: 'no_executor', task_id: t.id, message: `«${t.description}»: $
{why}` }); continue; }

```

```

    const s = scheduleByTask.get(t.id);
    if (s && t.deadline && s.end_at > t.deadline) {
        errors.push({ code: 'late', task_id: t.id, message: `«${t.description}»: по загрузке сотру
дников закончится ${fmtDate(s.end_at)}, а срок – ${fmtDate(t.deadline)}. Измените срок, приори
тет или исполнителя` });
    }
}

// Порядок: ремонт не может начаться до окончания осмотра/мойки этого ТС (при ручной фиксации)
for (const a of active) {
    const sa = scheduleByTask.get(a.id);
    if (!sa) continue;
    for (const b of active) {
        if (b.vehicle_type !== a.vehicle_type || b.vehicle_id !== a.vehicle_id || b.seq >= a.seq)
continue;
        const sb = scheduleByTask.get(b.id);
        if (sb && sa.start_at < sb.end_at) {
            errors.push({ code: 'sequence', task_id: a.id, message: `«${a.description}» начинается р
аньше окончания «${b.description}» (сначала мойка/осмотр, потом ремонт)` });
            break;
        }
    }
}
return errors;
}

// ===== Файл: server/domain/scheduler.js =====
// Автоматическое распределение задач между сотрудниками и построение расписания.
//
// Алгоритм – списочное планирование в рабочих минутах:
// 1. Задачи, которые уже выполняются / закреплены вручную / не входят в пересчёт, – фиксирован
ная загрузка.
// 2. Готовой к постановке считается задача, у которой все предшественники (мойка → осмотр → ре
монт
→ регулировки по тому же ТС в том же ремонте) уже поставлены.
// 3. Из готовых берётся самая важная (эффективный приоритет с учётом последователей, затем сро
к).
// 4. Для каждого допущенного сотрудника считается длительность (норма × его средний коэффициен
т
времени) и самое раннее окно, где свободен и сотрудник, и ТС (не более N человек на одном
ТС).
// 5. Выбирается сотрудник с самым ранним окончанием; профильная специализация даёт преимущес
тв
о.
import { calendarFor, getSettings } from '../settings.js';
import { json } from '../db.js';

const ROUND = 5;
const ceilTo = (v, step) => Math.ceil(v / step) * step;

```

```

export function loadEmployees(db) {
  const emps = db.prepare('SELECT * FROM employees WHERE active = 1').all()
    .map((e) => ({ ...e, specializations: json(e.specializations, []) }));
  const access = new Map();
  for (const r of db.prepare('SELECT work_type_id, employee_id FROM work_type_access').all()) {
    if (!access.has(r.work_type_id)) access.set(r.work_type_id, new Set());
    access.get(r.work_type_id).add(r.employee_id);
  }
  return { emps, access };
}

// Кто допущен к работе
export function eligibleEmployees(wt, emps, access) {
  if (!wt) return emps.filter((e) => e.general_access && !e.is_external);
  const allowed = access.get(wt.id) || new Set();
  if (wt.external) return emps.filter((e) => e.is_external || allowed.has(e.id));
  if (wt.required_qualification) {
    const q = wt.required_qualification.toLowerCase();
    return emps.filter((e) => allowed.has(e.id) || `${e.qualification} || ` ${e.position} || `
    ` .toLowerCase().includes(q));
  }
  if (wt.access_all) return emps.filter((e) => (e.general_access && !e.is_external) || allowed.h
  as(e.id));
  return emps.filter((e) => allowed.has(e.id));
}

// Средний коэффициент «факт / норма» по категориям работ (по выполненным задачам)
export function speedStats(db) {
  const rows = db.prepare(`SELECT t.employee_id, w.category, COUNT(*) n, AVG(t.actual_hours / t.
  hours) k
  FROM tasks t JOIN work_types w ON w.id = t.work_type_id
  WHERE t.status = 'done' AND t.actual_hours > 0 AND t.hours > 0 AND t.employee_id IS NOT NULL
  GROUP BY t.employee_id, w.category`).all();
  const m = new Map();
  for (const r of rows) m.set(`${r.employee_id}|${r.category}`, r);
  return m;
}

export function factorFor(emp, category, stats) {
  const s = stats.get(`${emp.id}|${category}`);
  if (s && s.n >= 3) return Math.min(2.5, Math.max(0.5, s.k));
  return emp.speed_factor || 1;
}

const overlaps = (a0, a1, b0, b1) => a0 < b1 && b0 < a1;

function maxConcurrent(intervals, s, e) {
  const pts = [];
  for (const [a, b] of intervals) if (overlaps(a, b, s, e)) { pts.push([Math.max(a, s), 1], [Mat
  h.min(b, e), -1]); }
}

```

```

    pts.sort((x, y) => x[0] - y[0] || x[1] - y[1]);
    let cur = 0, max = 0;
    for (const [, d] of pts) { cur += d; max = Math.max(max, cur); }
    return max;
}

function earliestSlot({ est, dur, empBusy, vehBusy, maxVeh, checkEmp }) {
    const cands = new Set([est]);
    if (checkEmp) for (const [, b] of empBusy) if (b > est) cands.add(b);
    for (const [, b] of vehBusy) if (b > est) cands.add(b);
    for (const c of [...cands].sort((a, b) => a - b)) {
        const e = c + dur;
        if (checkEmp && empBusy.some(([a, b]) => overlaps(a, b, c, e))) continue;
        if (maxVeh > 0 && maxConcurrent(vehBusy, c, e) >= maxVeh) continue;
        return c;
    }
    return null;
}

/**
 * Пересчитать расписание.
 * @param opts.orderIds – пересчитать только задачи этих ремонтов (остальные – фиксированная заг
рузка); null = все
 * @param opts.now – момент «сейчас»
 * @returns { placed: Map(taskId → {employee_id,start_at,end_at}), unschedulable: Map(taskId → п
ричина) }
 */
export function rebuildSchedule(db, { orderIds = null, now = new Date() } = {}) {
    const settings = getSettings(db);
    const cal = calendarFor(db);
    const maxVeh = Number(settings.max_workers_per_vehicle) || 0;
    const { emps, access } = loadEmployees(db);
    const empById = new Map(emps.map((e) => [e.id, e]));
    const wts = new Map(db.prepare('SELECT * FROM work_types').all().map((w) => [w.id, w]));
    const stats = speedStats(db);
    const nowW = ceilTo(cal.toWork(now), ROUND);

    const activeOrders = db.prepare(`SELECT id FROM repair_orders WHERE status IN ('planned','plan
_error','approved','in_progress')`).all().map((r) => r.id);
    const target = new Set(orderIds ? orderIds.filter((id) => activeOrders.includes(id)) : activeO
rders);

    const allTasks = db.prepare(`SELECT t.*, s.employee_id AS s_emp, s.start_at AS s_start, s.end_
at AS s_end, s.locked AS s_locked
    FROM tasks t LEFT JOIN schedule s ON s.task_id = t.id
    WHERE t.status IN ('planned','in_progress','paused')`).all();

    const empBusy = new Map(emps.map((e) => [e.id, []]));
    const vehBusy = new Map();
    const vkey = (t) => `${t.vehicle_type}:${t.vehicle_id}`;

```

```

const addBusy = (t, empId, s, e) => {
  if (empBusy.has(empId) && !empById.get(empId)?.is_external) empBusy.get(empId).push([s, e]);
  if (!vehBusy.has(vkey(t))) vehBusy.set(vkey(t), []);
  vehBusy.get(vkey(t)).push([s, e]);
};

const toPlace = [];
const fixedEnd = new Map(); // taskId → work-minute end (для предшественников)
for (const t of allTasks) {
  const movable = target.has(t.order_id) && t.status === 'planned' && !t.s_locked;
  if (movable) { toPlace.push(t); continue; }
  if (t.s_start && t.s_end && t.s_emp) {
    const s = cal.toWork(t.s_start);
    let e = cal.toWork(t.s_end);
    // выполняется дольше плана – сотрудник и ТС заняты как минимум до «сейчас»
    if (t.status !== 'planned' && e < nowW) e = nowW + ROUND;
    addBusy(t, t.s_emp, s, e);
    fixedEnd.set(t.id, e);
  }
}

// Предшественники: та же заявка, то же ТС, меньший seq
const groups = new Map();
for (const t of allTasks) {
  const g = `${t.order_id}|${vkey(t)}`;
  if (!groups.has(g)) groups.set(g, []);
  groups.get(g).push(t);
}

const preds = new Map();
const effPrio = new Map();
const effDeadline = new Map();
for (const t of toPlace) {
  const g = groups.get(`${t.order_id}|${vkey(t)}`) || [];
  preds.set(t.id, g.filter((x) => x.seq < t.seq).map((x) => x.id));
  const succ = g.filter((x) => x.seq >= t.seq);
  effPrio.set(t.id, Math.min(...succ.map((x) => x.priority)));
  effDeadline.set(t.id, succ.map((x) => x.deadline).filter(Boolean).sort()[0] || '9999');
}

const placed = new Map();
const unschedulable = new Map();
const endOf = (id) => (placed.has(id) ? placed.get(id).endW : fixedEnd.get(id));
const remaining = new Set(toPlace.map((t) => t.id));
const byId = new Map(toPlace.map((t) => [t.id, t]));

while (remaining.size) {
  const ready = [...remaining].map((id) => byId.get(id)).filter((t) => preds.get(t.id).every((p) => !remaining.has(p)));
  if (!ready.length) { // защита от цикла
    for (const id of remaining) unschedulable.set(id, 'не удалось упорядочить задачи');
  }
}

```

```

    break;
  }
  ready.sort((a, b) => effPrio.get(a.id) - effPrio.get(b.id)
    || effDeadline.get(a.id).localeCompare(effDeadline.get(b.id))
    || a.order_id - b.order_id || a.seq - b.seq || a.id - b.id);
  const t = ready[0];
  remaining.delete(t.id);

  const predEnds = preds.get(t.id).map(endOf).filter((x) => x !== null);
  const est = Math.max(nowW, ...predEnds);
  const wt = wts.get(t.work_type_id);
  let candS = eligibleEmployees(wt, emps, access);
  if (t.assigned_manual && t.employee_id) {
    const m = empById.get(t.employee_id);
    candS = m ? [m] : [];
  }
  if (!candS.length) {
    unschedulable.set(t.id, t.assigned_manual ? 'назначенный сотрудник неактивен' : 'нет сотру
дников с допуском к этой работе – настройте допуски в справочнике');
    continue;
  }

  let best = null;
  for (const emp of candS) {
    const factor = factorFor(emp, wt?.category, stats);
    const dur = Math.max(ROUND, ceilTo((t.hours || 1) * 60 * factor, ROUND));
    const s = earliestSlot({
      est, dur, empBusy: empBusy.get(emp.id) || [], vehBusy: vehBusy.get(vkey(t)) || [],
      maxVeh, checkEmp: !emp.is_external,
    });
    if (s == null) continue;
    const spec = wt && emp.specializations.includes(wt.category);
    // внешний сервис – только если сотрудники не справляются: штраф 1 рабочий день
    const score = s + dur + (spec ? 0 : 60) - (emp.level || 2) * 5 + (emp.is_external && !wt.is_
external ? cal.perDay : 0);
    if (!best || score < best.score) best = { emp, s, dur, score };
  }
  if (!best) { unschedulable.set(t.id, 'не найдено свободное окно'); continue; }
  addBusy(t, best.emp.id, best.s, best.s + best.dur);
  placed.set(t.id, {
    employee_id: best.emp.id, endW: best.s + best.dur,
    start_at: cal.fromWork(best.s), end_at: cal.fromWork(best.s + best.dur, true),
  });
}

// Запись результата
const tx = db.transaction(() => {
  const upsert = db.prepare(`INSERT INTO schedule(task_id, employee_id, vehicle_type, vehicle_
id, start_at, end_at, locked)
    VALUES (@task_id, @employee_id, @vehicle_type, @vehicle_id, @start_at, @end_at, 0)`);

```

```

    ON CONFLICT(task_id) DO UPDATE SET employee_id = excluded.employee_id, start_at = excluded
.start_at, end_at = excluded.end_at`);
    const setEmp = db.prepare('UPDATE tasks SET employee_id = ?, updated_at = ? WHERE id = ?');
    const del = db.prepare('DELETE FROM schedule WHERE task_id = ?');
    const ts = new Date().toISOString();
    for (const t of toPlace) {
        const p = placed.get(t.id);
        if (!p) { del.run(t.id); if (!t.assigned_manual) setEmp.run(null, ts, t.id); continue; }
        upsert.run({ task_id: t.id, employee_id: p.employee_id, vehicle_type: t.vehicle_type, vehi
cle_id: t.vehicle_id, start_at: p.start_at, end_at: p.end_at });
        if (t.employee_id !== p.employee_id) setEmp.run(p.employee_id, ts, t.id);
    }
});
tx();
return { placed, unschedulable, orderIds: [...target] };
}

```

```
// ===== Файл: server/domain/calendar.js =====
```

```
// Рабочий календарь. Время переводится в «рабочие минуты» – сквозной счётчик минут рабочего
// времени от фиксированной эпохи. В этом пространстве задачи, переходящие через ночь/выходные,
// – просто отрезки [start, end), что сильно упрощает поиск свободных окон.
```

```
const DAY = 86400000;
```

```
const EPOCH = Date.UTC(2020, 0, 6); // понедельник
```

```
const toMin = (hhmm) => {
    const [h, m] = String(hhmm).split(':').map(Number);
    return h * 60 + (m || 0);
};
```

```
export class WorkCalendar {
    constructor(settings) {
        this.offset = Number(settings.tz_offset_min ?? 180) * 60000;
        const ws = toMin(settings.work_start || '08:00');
        const we = toMin(settings.work_end || '17:00');
        const ls = settings.lunch_start ? toMin(settings.lunch_start) : null;
        const le = settings.lunch_end ? toMin(settings.lunch_end) : null;
        this.segments = ls != null && le != null && ls > ws && le < we && le > ls
            ? [[ws, ls], [le, we]]
            : [[ws, we]];
        this.perDay = this.segments.reduce((a, [s, e]) => a + (e - s), 0);
        this.workDays = new Set((settings.work_days || [1, 2, 3, 4, 5]).map(Number));
        this.holidays = new Set(settings.holidays || []);
    }

```

```

    isWorkDay(dayIdx) {
        const d = new Date(EPOCH + dayIdx * DAY);
        const dow = d.getUTCDay() || 7; // 1..7
        if (!this.workDays.has(dow)) return false;
    }

```

```

    return !this.holidays.has(d.toISOString().slice(0, 10));
}

// Число рабочих дней в [0, dayIdx)
workDaysBefore(dayIdx) {
    if (dayIdx <= 0) return 0;
    const p = this._prefix || (this._prefix = []);
    while (p.length <= dayIdx) {
        const i = p.length - 1;
        p.push(p[i] + (this.isWorkDay(i) ? 1 : 0));
    }
    return p[dayIdx];
}

// Реальная дата (Date|ISO) → рабочая минута (время вне рабочего окна сдвигается вперёд)
toWork(date) {
    const local = new Date(date).getTime() + this.offset;
    if (local < EPOCH) return 0;
    const dayIdx = Math.floor((local - EPOCH) / DAY);
    const minOfDay = Math.floor((local - EPOCH - dayIdx * DAY) / 60000);
    const base = this.workDaysBefore(dayIdx) * this.perDay;
    if (!this.isWorkDay(dayIdx)) return base;
    let acc = 0;
    for (const [s, e] of this.segments) {
        if (minOfDay < s) return base + acc;
        if (minOfDay < e) return base + acc + (minOfDay - s);
        acc += e - s;
    }
    return base + acc; // после конца дня → начало следующего рабочего дня
}

// Рабочая минута → реальная дата (ISO). atEnd=true: конец отрезка ставим на конец предыдущего
// окна
fromWork(wm, atEnd = false) {
    let workDay = Math.floor(wm / this.perDay);
    let within = wm - workDay * this.perDay;
    if (atEnd && within === 0 && workDay > 0) { workDay -= 1; within = this.perDay; }
    // найти календарный день с индексом workDay среди рабочих
    let lo = workDay, hi = workDay * 2 + 30;
    while (this.workDaysBefore(hi) <= workDay) hi *= 2;
    while (lo < hi) { // минимальный d: workDaysBefore(d+1) > workDay
        const mid = Math.floor((lo + hi) / 2);
        if (this.workDaysBefore(mid + 1) > workDay) hi = mid; else lo = mid + 1;
    }
    const dayIdx = lo;
    let minute = null;
    for (const [s, e] of this.segments) {
        const len = e - s;
        if (within < len || (atEnd && within === len)) { minute = s + within; break; }
        within -= len;
    }

```

```

    }
    if (minute == null) minute = this.segments[this.segments.length - 1][1];
    return new Date(EPOCH + dayIdx * DAY + minute * 60000 - this.offset).toISOString();
}

// Конец рабочего дня через n рабочих дней от даты (для сроков по приоритету)
addWorkDays(date, n) {
    const start = this.toWork(date);
    const day = Math.floor(start / this.perDay) + n;
    return this.fromWork((day + 1) * this.perDay, true);
}

// Начало/конец «локальных» суток для отображения
localDayStart(date) {
    const local = new Date(date).getTime() + this.offset;
    return new Date(Math.floor(local / DAY) * DAY - this.offset).toISOString();
}
}

// ===== Файл: server/domain/maintenance.js =====
// Учёт пробега и плановое ТО: статус по каждому правилу, прогноз даты по среднесуточному пробегу.
const DAY = 86400000;

export function vehicleRow(db, type, id) {
    return db.prepare(`SELECT * FROM ${type === 'tractor' ? 'tractors' : 'trailers'} WHERE id = ?`)
        .get(id);
}

export function rulesFor(db, type, vehicle) {
    const all = db.prepare('SELECT * FROM maintenance_rules WHERE active = 1 AND vehicle_type = ? ORDER BY id').all(type);
    const hay = `${vehicle.brand} || '|' ${vehicle.model} || '|'`.toUpperCase();
    const byKey = new Map();
    for (const r of all) {
        if (r.brand_match && !hay.includes(r.brand_match.toUpperCase())) continue;
        const prev = byKey.get(r.key);
        // правило под конкретную марку важнее общего
        if (!prev || (!prev.brand_match && r.brand_match)) byKey.set(r.key, r);
    }
    return [...byKey.values()];
}

// Среднесуточный пробег за последние 120 дней (км/сутки) по журналу пробега
export function dailyMileage(db, type, id) {
    const rows = db.prepare(`SELECT mileage, recorded_at FROM mileage_log
        WHERE vehicle_type = ? AND vehicle_id = ? AND recorded_at >= ? ORDER BY recorded_at`)
        .all(type, id, new Date(Date.now() - 120 * DAY).toISOString());
    if (rows.length < 2) return null;

```

```

    const first = rows[0], last = rows[rows.length - 1];
    const days = (new Date(last.recorded_at) - new Date(first.recorded_at)) / DAY;
    if (days < 3 || last.mileage <= first.mileage) return null;
    return Math.round((last.mileage - first.mileage) / days);
}

export function ruleStatus(db, type, vehicle, rule, now = new Date()) {
    const lastTask = rule.work_type_id ? db.prepare(`SELECT mileage, finished_at FROM tasks
        WHERE vehicle_type = ? AND vehicle_id = ? AND work_type_id = ? AND status = 'done'
        ORDER BY finished_at DESC LIMIT 1`).get(type, vehicle.id, rule.work_type_id) : null;
    const base = db.prepare('SELECT done_mileage, done_at FROM maintenance_baseline WHERE rule_id
    = ? AND vehicle_type = ? AND vehicle_id = ?')
        .get(rule.id, type, vehicle.id);

    let lastKm = null, lastAt = null, source = null;
    const cands = [];
    if (lastTask) cands.push({ km: lastTask.mileage, at: lastTask.finished_at, source: 'repair' })
    ;
    if (base) cands.push({ km: base.done_mileage, at: base.done_at, source: 'baseline' });
    cands.sort((a, b) => String(b.at || '').localeCompare(String(a.at || '')));
    if (cands.length) ({ km: lastKm, at: lastAt, source } = cands[0]);

    const mileage = vehicle.mileage || 0;
    let estimated = false;
    if (lastKm == null && rule.interval_km && mileage > 0) {
        lastKm = Math.floor(mileage / rule.interval_km) * rule.interval_km;
        estimated = true;
        source = 'estimate';
    }

    const nextKm = rule.interval_km && lastKm != null ? lastKm + rule.interval_km : null;
    const remainingKm = nextKm != null ? nextKm - mileage : null;
    const nextDate = rule.interval_days && lastAt ? new Date(new Date(lastAt).getTime() + rule.int
    erval_days * DAY).toISOString() : null;
    const daysLeft = nextDate ? Math.floor((new Date(nextDate) - now) / DAY) : null;

    const perDay = dailyMileage(db, type, vehicle.id);
    const forecastDate = remainingKm != null && perDay ? new Date(now.getTime() + Math.max(0, rema
    iningKm / perDay) * DAY).toISOString() : null;
    const dueDate = [nextDate, forecastDate].filter(Boolean).sort()[0] || null;

    let status = 'ok';
    if (remainingKm == null && daysLeft == null) status = 'unknown';
    else if ((remainingKm != null && remainingKm <= 0) || (daysLeft != null && daysLeft <= 0)) sta
    tus = 'overdue';
    else if ((remainingKm != null && remainingKm <= rule.warn_km) || (daysLeft != null && daysLeft
    <= rule.warn_days)) status = 'soon';

    return {
        rule_id: rule.id, key: rule.key, name: rule.name, work_type_id: rule.work_type_id,

```

```

    interval_km: rule.interval_km, interval_days: rule.interval_days,
    last_km: lastKm, last_at: lastAt, source, estimated,
    next_km: nextKm, remaining_km: remainingKm, next_date: nextDate, days_left: daysLeft,
    km_per_day: perDay, forecast_date: forecastDate, due_date: dueDate, status,
  };
}

export function vehicleMaintenance(db, type, vehicle, now) {
  return rulesFor(db, type, vehicle).map((r) => ruleStatus(db, type, vehicle, r, now));
}

export function fleetMaintenance(db, now = new Date()) {
  const out = [];
  for (const type of ['tractor', 'trailer']) {
    const rows = db.prepare(`SELECT * FROM ${type} === 'tractor' ? 'tractors' : 'trailers' WHERE
    archived = 0`).all();
    for (const v of rows) {
      for (const st of vehicleMaintenance(db, type, v, now)) {
        out.push({ vehicle_type: type, vehicle_id: v.id, plate: v.plate, brand: v.brand, model:
        v.model, mileage: v.mileage, ...st });
      }
    }
  }
  const order = { overdue: 0, soon: 1, unknown: 2, ok: 3 };
  return out.sort((a, b) => order[a.status] - order[b.status] || (a.remaining_km ?? 1e9) - (b.re
  maining_km ?? 1e9));
}

// Запись пробега: журнал + обновление текущего пробега ТС (пробег не уменьшается без явного фла
  га)
export function recordMileage(db, { type, id, mileage, at, source, refId, userId, allowDecrease
  = false }) {
  if (mileage == null || Number.isNaN(Number(mileage))) return;
  const m = Math.round(Number(mileage));
  const table = type === 'tractor' ? 'tractors' : 'trailers';
  const v = db.prepare(`SELECT mileage FROM ${table} WHERE id = ?`).get(id);
  if (!v) return;
  db.prepare(`INSERT INTO mileage_log(vehicle_type, vehicle_id, mileage, recorded_at, source, re
  f_id, user_id)
  VALUES (?, ?, ?, ?, ?, ?, ?)`).run(type, id, m, at || new Date().toISOString(), source, refI
  d ?? null, userId ?? null);
  if (m > v.mileage || allowDecrease) {
    db.prepare(`UPDATE ${table} SET mileage = ?, updated_at = ? WHERE id = ?`).run(m, new Date()
    .toISOString(), id);
  }
}

// ===== Файл: server/services/orders.js =====
import { json, nowIso } from '../db.js';

```

```

import { calendarFor, getSettings } from '../settings.js';
import { generatePlan, validatePlan } from '../domain/planner.js';
import { rebuildSchedule } from '../domain/scheduler.js';
import { recordMileage } from '../domain/maintenance.js';
import { bad, fmtDateTime, notFound } from '../util.js';

export function nextOrderNumber(db) {
  const year = new Date().getFullYear();
  const row = db.prepare('SELECT number FROM repair_orders WHERE number LIKE ? ORDER BY id DESC LIMIT 1').get(`${year}-%`);
  const n = row ? Number(row.number.split('-')[1]) + 1 : 1;
  return `${year}-${String(n).padStart(4, '0')}`;
}

const getTractor = (db, id) => (id ? db.prepare('SELECT * FROM tractors WHERE id = ?').get(id) : null);
const getTrailer = (db, id) => (id ? db.prepare('SELECT * FROM trailers WHERE id = ?').get(id) : null);

export function previewPlan(db, input) {
  const tractor = getTractor(db, input.tractor_id);
  const trailer = getTrailer(db, input.trailer_id);
  if (!tractor && !trailer) throw bad('Выберите тягач и/или прицеп');
  return generatePlan(db, {
    tractor, trailer, cal: calendarFor(db),
    defects: input.defects || {},
    inspected_at: input.inspected_at || nowIso(),
    release_deadline: input.release_deadline || null,
    tractor_mileage: input.tractor_mileage ?? null,
    trailer_mileage: input.trailer_mileage ?? null,
  });
}

function insertTasks(db, orderId, tasks) {
  const ins = db.prepare('INSERT INTO tasks(order_id, vehicle_type, vehicle_id, work_type_id, description, location, reason, source, seq, priority, deadline, hours, price, salary, mileage, status, registered_at) VALUES (@order_id, @vehicle_type, @vehicle_id, @work_type_id, @description, @location, @reason, @source, @seq, @priority, @deadline, @hours, @price, @salary, @mileage, 'planned', @registered_at)`);
  const ts = nowIso();
  for (const t of tasks) {
    ins.run({
      order_id: orderId, vehicle_type: t.vehicle_type, vehicle_id: t.vehicle_id, work_type_id: t.work_type_id,
      description: t.description, location: t.location, reason: t.reason, source: t.source, seq: t.seq,
      priority: t.priority, deadline: t.deadline, hours: t.hours, price: t.price ?? null, salary: t.salary ?? null,
    });
  }
}

```

```

        mileage: t.mileage ?? null, registered_at: ts,
    });
}
}

/** Создать ремонт из дефектовки: план ремонта + расписание. */
export function createOrder(db, input, user) {
    const plan = previewPlan(db, input);
    const tx = db.transaction(() => {
        const number = nextOrderNumber(db);
        const inspectedAt = input.inspected_at || nowIso();
        const res = db.prepare(`INSERT INTO repair_orders(number, tractor_id, trailer_id, tractor_mileage, trailer_mileage, reefer_hours, inspected_at, release_deadline, status, defects, inspection, notes, created_by) VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'planned', ?, ?, ?, ?)`).run(number, input.tractor_id || null, input.trailer_id || null, input.tractor_mileage ?? null, input.trailer_mileage ?? null, input.reefer_hours ?? null, inspectedAt, input.release_deadline || null, JSON.stringify(input.defects || {}), JSON.stringify({ zones: plan.inspection, damages: plan.damages, recommendations: plan.recommendations, warnings: plan.warnings }), input.notes || null, user?.id ?? null, );
        const orderId = Number(res.lastInsertRowid);
        insertTasks(db, orderId, plan.tasks);
        if (input.tractor_id && input.tractor_mileage) {
            recordMileage(db, { type: 'tractor', id: input.tractor_id, mileage: input.tractor_mileage, at: inspectedAt, source: 'defect', refId: orderId, userId: user?.id });
        }
        if (input.trailer_id && input.trailer_mileage) {
            recordMileage(db, { type: 'trailer', id: input.trailer_id, mileage: input.trailer_mileage, at: inspectedAt, source: 'defect', refId: orderId, userId: user?.id });
        }
        if (input.trailer_id && input.reefer_hours) {
            db.prepare('UPDATE trailers SET reefer_hours = ? WHERE id = ?').run(input.reefer_hours, input.trailer_id);
        }
        // если прицеп сцеплен с другим тягачом – обновить сцепку
        if (input.tractor_id && input.trailer_id) {
            db.prepare('UPDATE tractors SET trailer_id = NULL WHERE trailer_id = ? AND id <> ?').run(input.trailer_id, input.tractor_id);
            db.prepare('UPDATE tractors SET trailer_id = ? WHERE id = ?').run(input.trailer_id, input.tractor_id);
        }
        audit(db, user, 'order', orderId, 'create', `Ремонт ${number}: ${plan.tasks.length} работ`);
        return orderId;
    });
    const orderId = tx();
    replan(db, orderId);
    return orderId;
}

```

```

}

/** Пересчитать расписание по ремонту (или по всем) и сохранить ошибки планирования. */
export function replan(db, orderId = null, { now = new Date() } = {}) {
  const result = rebuildSchedule(db, { orderIds: orderId ? [orderId] : null, now });
  const ids = orderId ? [orderId] : result.orderIds;
  for (const id of ids) refreshOrderState(db, id, { unschedulable: result.unschedulable, now });
  return result;
}

export function refreshOrderState(db, orderId, { unschedulable = new Map(), now = new Date() } =
  {}) {
  const order = db.prepare('SELECT * FROM repair_orders WHERE id = ?').get(orderId);
  if (!order) return null;
  const tasks = db.prepare('SELECT * FROM tasks WHERE order_id = ?').all(orderId);
  const sched = db.prepare('SELECT s.* FROM schedule s JOIN tasks t ON t.id = s.task_id WHERE t.
    order_id = ?').all(orderId);
  const scheduleByTask = new Map(sched.map((s) => [s.task_id, s]));
  const off = Number(getSettings(db).tz_offset_min ?? 180);
  // задачи без строки расписания, но в статусе planned – тоже ошибка
  for (const t of tasks) {
    if (t.status === 'planned' && !scheduleByTask.has(t.id) && !unschedulable.has(t.id)
      && ['planned', 'plan_error', 'approved', 'in_progress'].includes(order.status)) {
      unschedulable.set(t.id, 'не поставлена в расписание – нажмите «Пересчитать расписание»');
    }
  }
  const errors = validatePlan(order, tasks, { scheduleByTask, unschedulable, now, fmtDate: (iso)
    => fmtDateTime(iso, off) });

  let status = order.status;
  const live = tasks.filter((t) => t.status !== 'cancelled');
  if (['planned', 'plan_error', 'approved', 'in_progress'].includes(status)) {
    if (live.length && live.every((t) => t.status === 'done')) status = 'done';
    else if (live.some((t) => ['in_progress', 'paused', 'done'].includes(t.status))) status = 'i
n_progress';
    else if (status !== 'approved') status = errors.length ? 'plan_error' : 'planned';
  }
  if (status === 'done' && order.status !== 'done') {
    db.prepare('UPDATE repair_orders SET closed_at = ? WHERE id = ?').run(nowIso(), orderId);
  }
  db.prepare('UPDATE repair_orders SET plan_errors = ?, status = ?, updated_at = ? WHERE id = ?'
  )
    .run(JSON.stringify(errors), status, nowIso(), orderId);
  return { status, errors };
}

export function loadOrder(db, id) {
  const o = db.prepare(`SELECT o.*, tr.plate AS tractor_plate, tr.brand AS tractor_brand, tr.mod
    el AS tractor_model,
    tl.plate AS trailer_plate, tl.brand AS trailer_brand, tl.model AS trailer_model, u.name AS

```

```

    created_by_name
    FROM repair_orders o
    LEFT JOIN tractors tr ON tr.id = o.tractor_id
    LEFT JOIN trailers tl ON tl.id = o.trailer_id
    LEFT JOIN users u ON u.id = o.created_by
    WHERE o.id = ?`).get(id);
if (!o) throw notFound('Ремонт');
o.defects = json(o.defects, {});
o.inspection = json(o.inspection, {});
o.plan_errors = json(o.plan_errors, []);
o.tasks = db.prepare(`SELECT t.*, w.category, w.code AS work_code, e.name AS employee_name,
    s.start_at, s.end_at, s.locked, s.employee_id AS scheduled_employee_id
    FROM tasks t
    LEFT JOIN work_types w ON w.id = t.work_type_id
    LEFT JOIN employees e ON e.id = t.employee_id
    LEFT JOIN schedule s ON s.task_id = t.id
    WHERE t.order_id = ? ORDER BY t.seq, t.priority, COALESCE(s.start_at, '9999'), t.id`).all(id
);
o.parts = db.prepare('SELECT * FROM installed_parts WHERE order_id = ? ORDER BY installed_at')
.all(id);
return o;
}

export function audit(db, user, entity, entityId, action, details) {
    db.prepare('INSERT INTO audit_log(user_id, entity, entity_id, action, details) VALUES (?, ?, ?
, ?, ?)')
        .run(user?.id ?? null, entity, entityId ?? null, action, details ?? null);
}

// ===== Файл: server/services/reports.js =====
// Отчёты: построение табличных данных и выгрузка в Excel / PDF / CSV.
import ExcelJS from 'exceljs';
import PDFDocument from 'pdfkit';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { fleetMaintenance } from '../domain/maintenance.js';
import { PRIORITIES } from '../domain/checklist.js';
import { getSettings } from '../settings.js';
import { bad, fmtDateTime } from '../util.js';

const here = path.dirname(fileURLToPath(import.meta.url));
const FONT = path.join(here, '..', 'fonts', 'DejaVuSans.ttf');
const FONT_BOLD = path.join(here, '..', 'fonts', 'DejaVuSans-Bold.ttf');

const STATUS = { planned: 'Запланирована', in_progress: 'В работе', paused: 'Приостановлена', do
ne: 'Выполнена', cancelled: 'Отменена' };
const ORDER_STATUS = { draft: 'Черновик', planned: 'Запланирован', plan_error: 'Ошибка планирова
ния', approved: 'Утверждён', in_progress: 'В ремонте', done: 'Завершён', cancelled: 'Отменён'
};

```

```

const MAINT = { overdue: 'Просрочено', soon: 'Скоро', ok: 'В норме', unknown: 'Нет данных' };

export const REPORTS = {
  works: 'Выполненные работы',
  salary: 'Зарплата и выработка сотрудников',
  parts: 'Использованные запчасти',
  vehicles: 'Состояние тягачей и прицепов',
  orders: 'Ремонты за период',
};

const sum = (rows, k) => rows.reduce((a, r) => a + (Number(r[k]) || 0), 0);
const r2 = (v) => Math.round(v * 100) / 100;

export function buildReport(db, type, q) {
  const off = Number(getSettings(db).tz_offset_min ?? 180);
  const from = q.from || '1970-01-01';
  const to = q.to || '9999-12-31';
  const period = `${q.from ? fmtDateTime(q.from, off, false) : 'начало учёта'} – ${q.to ? fmtDateTime(new Date(new Date(q.to).getTime() - 1).toISOString(), off, false) : 'сегодня'}`;
  const vehFilter = q.vehicle_type && q.vehicle_id ? ' AND t.vehicle_type = @vt AND t.vehicle_id = @vid' : '';
  const p = { from, to, vt: q.vehicle_type, vid: Number(q.vehicle_id) || null, emp: Number(q.employee_id) || null, cat: q.category || null };
  const plateSql = `CASE t.vehicle_type WHEN 'tractor' THEN tr.plate ELSE tl.plate END`;
  const vehJoin = `LEFT JOIN tractors tr ON t.vehicle_type = 'tractor' AND tr.id = t.vehicle_id
    LEFT JOIN trailers tl ON t.vehicle_type = 'trailer' AND tl.id = t.vehicle_id`;

  if (type === 'works') {
    const rows = db.prepare(`SELECT t.finished_at, o.number, ${plateSql} AS plate, t.vehicle_type, t.description, t.location, w.category,
      e.name AS employee, t.hours, t.actual_hours, t.mileage, t.price, t.salary
    FROM tasks t LEFT JOIN repair_orders o ON o.id = t.order_id LEFT JOIN employees e ON e.id = t.employee_id
    LEFT JOIN work_types w ON w.id = t.work_type_id ${vehJoin}
    WHERE t.status = 'done' AND t.finished_at >= @from AND t.finished_at < @to ${vehFilter}
    ${p.emp ? 'AND t.employee_id = @emp' : ''} ${p.cat ? 'AND w.category = @cat' : ''}
    ORDER BY t.finished_at`).all(p).map((r) => ({ ...r, finished_at: fmtDateTime(r.finished_at, off), veh: r.vehicle_type === 'tractor' ? 'Тягач' : 'Прицеп' }));
    return {
      title: REPORTS.works, period,
      columns: [
        { key: 'finished_at', title: 'Дата', width: 15 }, { key: 'number', title: 'Ремонт', width: 10 },
        { key: 'plate', title: 'Госномер', width: 12 }, { key: 'description', title: 'Работа', width: 38 },
        { key: 'location', title: 'Узел', width: 16 }, { key: 'employee', title: 'Исполнитель', width: 14 },
        { key: 'hours', title: 'Норма, ч', type: 'num', width: 8 }, { key: 'actual_hours', title: 'Факт, ч', type: 'num', width: 8 },
        { key: 'mileage', title: 'Пробег, км', type: 'int', width: 10 },
      ],
    };
  }
}

```

```

        { key: 'price', title: 'Цена, ₺', type: 'money', width: 11 }, { key: 'salary', title: 'З
П, ₺', type: 'money', width: 10 },
    ],
    rows,
    totals: { description: `Итого работ: ${rows.length}`, hours: r2(sum(rows, 'hours')), actual_hours: r2(sum(rows, 'actual_hours')), price: sum(rows, 'price'), salary: sum(rows, 'salary')
    },
    };
}

```

```

if (type === 'salary') {
    const rows = db.prepare(`SELECT e.name AS employee, e.position, COUNT(t.id) AS tasks, COALESCE(SUM(t.hours),0) AS hours,
        COALESCE(SUM(t.actual_hours),0) AS actual_hours, COALESCE(SUM(t.price),0) AS price, COALESCE(SUM(t.salary),0) AS salary
    FROM employees e JOIN tasks t ON t.employee_id = e.id AND t.status = 'done' AND t.finished_at >= @from AND t.finished_at < @to ${vehFilter}
    ${p.emp ? 'WHERE e.id = @emp' : ''}
    GROUP BY e.id ORDER BY salary DESC`).all(p)
    .map((r) => ({ ...r, hours: r2(r.hours), actual_hours: r2(r.actual_hours), efficiency: r2(r.actual_hours ? Math.round((r.hours / r.actual_hours) * 100) : null })));
    return {
        title: REPORTS.salary, period,
        columns: [
            { key: 'employee', title: 'Сотрудник', width: 20 }, { key: 'position', title: 'Должность', width: 22 },
            { key: 'tasks', title: 'Работ', type: 'int', width: 8 }, { key: 'hours', title: 'Норма, ч', type: 'num', width: 10 },
            { key: 'actual_hours', title: 'Факт, ч', type: 'num', width: 10 }, { key: 'efficiency', title: 'Выработка, %', type: 'int', width: 12 },
            { key: 'price', title: 'Сумма работ, ₺', type: 'money', width: 14 }, { key: 'salary', title: 'Зарплата, ₺', type: 'money', width: 13 },
        ],
        rows,
        totals: { employee: 'Итого', tasks: sum(rows, 'tasks'), hours: r2(sum(rows, 'hours')), actual_hours: r2(sum(rows, 'actual_hours')), price: sum(rows, 'price'), salary: sum(rows, 'salary') },
    };
}

```

```

if (type === 'parts') {
    const vf = q.vehicle_type && q.vehicle_id ? ' AND ip.vehicle_type = @vt AND ip.vehicle_id = @vid' : '';
    const rows = db.prepare(`SELECT ip.installed_at, o.number, CASE ip.vehicle_type WHEN 'tractor' THEN tr.plate ELSE tl.plate END AS plate,
        ip.code, ip.article, ip.name, ip.unit, ip.qty, ip.price, ip.qty * COALESCE(ip.price,0) AS total, ip.mileage, ip.exported_at, ip.source
    FROM installed_parts ip LEFT JOIN repair_orders o ON o.id = ip.order_id
    LEFT JOIN tractors tr ON ip.vehicle_type = 'tractor' AND tr.id = ip.vehicle_id
    LEFT JOIN trailers tl ON ip.vehicle_type = 'trailer' AND tl.id = ip.vehicle_id

```

```

    WHERE ip.installed_at >= @from AND ip.installed_at < @to ${vf} ORDER BY ip.installed_at`).
    all(p)
    .map((r) => ({ ...r, installed_at: fmtDateTime(r.installed_at, off), exported: r.exported_
at ? fmtDateTime(r.exported_at, off) : 'нет', source: r.source === 'lc' ? 'LC' : 'вручную' })))
;
    return {
        title: REPORTS.parts, period,
        columns: [
            { key: 'installed_at', title: 'Дата', width: 15 }, { key: 'number', title: 'Ремонт', wid
th: 10 }, { key: 'plate', title: 'Госномер', width: 12 },
            { key: 'code', title: 'Код LC', width: 11 }, { key: 'article', title: 'Артикул', width:
13 }, { key: 'name', title: 'Наименование', width: 34 },
            { key: 'qty', title: 'Кол-во', type: 'num', width: 7 }, { key: 'unit', title: 'Ед.', wid
th: 5 }, { key: 'price', title: 'Цена, ₽', type: 'money', width: 10 },
            { key: 'total', title: 'Сумма, ₽', type: 'money', width: 11 }, { key: 'source', title: '
Источник', width: 9 }, { key: 'exported', title: 'B LC', width: 15 },
        ],
        rows,
        totals: { name: `Позиций: ${rows.length}`, total: sum(rows, 'total') },
    };
}

if (type === 'vehicles') {
    const maint = fleetMaintenance(db);
    const worst = new Map();
    const rank = { overdue: 0, soon: 1, unknown: 2, ok: 3 };
    for (const m of maint) {
        const k = `${m.vehicle_type}:${m.vehicle_id}`;
        const cur = worst.get(k);
        if (!cur || rank[m.status] < rank[cur.status]) worst.set(k, m);
    }
    const rows = [];
    for (const vt of ['tractor', 'trailer']) {
        if (q.vehicle_type && q.vehicle_type !== vt) continue;
        const list = db.prepare(`SELECT v.* FROM ${vt === 'tractor' ? 'tractors' : 'trailers'} v W
HERE v.archived = 0 ${q.vehicle_id ? 'AND v.id = @vid' : ''} ORDER BY v.plate`).all(p);
        for (const v of list) {
            const st = db.prepare(`SELECT
                (SELECT COUNT(*) FROM tasks WHERE vehicle_type = @vt AND vehicle_id = @id AND status
IN ('planned','in_progress','paused')) AS open_tasks,
                (SELECT COUNT(*) FROM tasks WHERE vehicle_type = @vt AND vehicle_id = @id AND status
= 'done' AND finished_at >= @from AND finished_at < @to) AS done_tasks,
                (SELECT COALESCE(SUM(price),0) FROM tasks WHERE vehicle_type = @vt AND vehicle_id =
@id AND status = 'done' AND finished_at >= @from AND finished_at < @to) AS works_sum,
                (SELECT COALESCE(SUM(qty * COALESCE(price,0)),0) FROM installed_parts WHERE vehicle_
type = @vt AND vehicle_id = @id AND installed_at >= @from AND installed_at < @to) AS parts_sum
            ,
                (SELECT MAX(finished_at) FROM tasks WHERE vehicle_type = @vt AND vehicle_id = @id AN
D status = 'done') AS last_repair`).get({ vt, id: v.id, from, to });
            const w = worst.get(`${vt}:${v.id}`);

```

```

export const useToast = () => useContext(ToastCtx);

export function ToastProvider({ children }) {
  const [items, setItems] = useState([]);
  const push = useCallback((text, tone = 'green') => {
    const id = Math.random();
    setItems((x) => [...x, { id, text, tone }]);
    setTimeout(() => setItems((x) => x.filter((i) => i.id !== id)), tone === 'red' ? 7000 : 3500
  );
}, []);
  return (
    <ToastCtx.Provider value={push}>
      {children}
      <div className="toasts" aria-live="polite">
        {items.map((t) => <div key={t.id} className={`toast toast-${t.tone}`}>{t.text}</div>)}
      </div>
    </ToastCtx.Provider>
  );
}

// ----- Загрузка данных -----
export function useLoad(fn, deps = []) {
  const [state, setState] = useState({ data: null, loading: true, error: null });
  const fnRef = useRef(fn);
  fnRef.current = fn;
  const reload = useCallback(async (silent = false) => {
    if (!silent) setState((s) => ({ ...s, loading: true, error: null }));
    try {
      const data = await fnRef.current();
      setState({ data, loading: false, error: null });
      return data;
    } catch (error) {
      setState((s) => ({ ...s, loading: false, error }));
      return null;
    }
  }, []);
  // eslint-disable-next-line react-hooks/exhaustive-deps
  useEffect(() => { reload(); }, deps);
  const setData = useCallback((data) => setState((s) => ({ ...s, data: typeof data === 'function'
    ? data(s.data) : data })), []);
  return { ...state, reload, setData };
}

// Выполнение действия с уведомлением об ошибке/офлайн-очереди
export function useAction() {
  const toast = useToast();
  const [busy, setBusy] = useState(false);
  const run = useCallback(async (fn, okText) => {
    setBusy(true);
    try {

```

```

    const res = await fn();
    if (res && res.queued) toast('Нет связи: изменение сохранено и будет отправлено автоматиче
ски', 'amber');
    else if (okText) toast(okText);
    return res;
  } catch (e) {
    toast(e.message || 'Ошибка', 'red');
    return null;
  } finally {
    setBusy(false);
  }
}, [toast]);
return [run, busy];
}

// ----- Выбор из списка с поиском -----
export function SearchSelect({ options, value, onChange, placeholder = 'Выберите...', groupBy, ren
der, disabled, clearable, autoFocus }) {
  const [open, setOpen] = useState(false);
  const [q, setQ] = useState('');
  const [hi, setHi] = useState(0);
  const ref = useRef(null);
  const inputRef = useRef(null);
  const selected = options.find((o) => o.value === value);
  const filtered = useMemo(() => {
    const words = q.toLowerCase().split(/\s+/).filter(Boolean);
    return options.filter((o) => {
      const hay = `${o.label} ${o.search || ''} ${groupBy ? o[groupBy] || '' : ''}`.toLowerCase(
);
      return words.every((w) => hay.includes(w));
    }).slice(0, 300);
  }, [options, q, groupBy]);

  useEffect(() => {
    const close = (e) => { if (ref.current && !ref.current.contains(e.target)) setOpen(false); }
    ;
    document.addEventListener('mousedown', close);
    return () => document.removeEventListener('mousedown', close);
  }, []);
  useEffect(() => { if (open) { setHi(0); setTimeout(() => inputRef.current?.focus(), 0); } }, [
open]);
  useEffect(() => { if (autoFocus) setOpen(true); }, [autoFocus]);

  const choose = (o) => { onChange(o ? o.value : null, o); setOpen(false); setQ(''); };
  const onKey = (e) => {
    if (e.key === 'ArrowDown') { e.preventDefault(); setHi((h) => Math.min(h + 1, filtered.length
h - 1)); }
    if (e.key === 'ArrowUp') { e.preventDefault(); setHi((h) => Math.max(h - 1, 0)); }
    if (e.key === 'Enter') { e.preventDefault(); if (filtered[hi]) choose(filtered[hi]); }
    if (e.key === 'Escape') setOpen(false);
  }
}

```

```

};

let lastGroup = null;
return (
  <div className={cx('ss', disabled && 'disabled')} ref={ref}>
    <button type="button" className="ss-trigger input" onClick={() => !disabled && setOpen((o) => !o)} disabled={disabled}>
      <span className={selected ? '' : 'muted'}>{selected ? (render ? render(selected) : selected.label) : placeholder}</span>
      <ChevronDown size={16} />
    </button>
    {open && (
      <div className="ss-pop">
        <div className="ss-search">
          <Search size={15} />
          <input ref={inputRef} value={q} onChange={(e) => { setQ(e.target.value); setHi(0); }}
        > onKeyDown={onKey} placeholder="Поиск..." />
        </div>
        <div className="ss-list">
          {clearable && value != null && <button type="button" className="ss-opt muted" onClick={() => choose(null)}>— не выбрано </button>}
          {filtered.length === 0 && <div className="ss-empty">Ничего не найдено</div>}
          {filtered.map((o, i) => {
            const g = groupBy ? o[groupBy] : null;
            const head = g && g !== lastGroup ? <div className="ss-group" key={`g-${g}`}>{g}</div> : null;
            lastGroup = g;
            return [head, (
              <button type="button" key={o.value} className={cx('ss-opt', i === hi && 'hi', o.value === value && 'sel')}
                onMouseEnter={() => setHi(i)} onClick={() => choose(o)}>
                <span>{render ? render(o) : o.label}</span>
                {o.value === value && <Check size={14} />}
              </button>
            )];
          })}
        </div>
      </div>
    )}
  </div>
);
}

```

```

export function Confirm({ title, text, onConfirm, onClose, danger, confirmText = 'Подтвердить' }) {
  const [busy, setBusy] = useState(false);
  return (
    <Modal title={title} onClose={onClose} footer={<>
      <Button onClick={onClose}>Отмена</Button>
      <Button variant={danger ? 'danger' : 'primary'} loading={busy} onClick={async () => { setB

```

```

    usy(true); try { await onConfirm(); } finally { setBusy(false); } onClose(); }}>{confirmText}<
  /Button>
    </>>
    <p>{text}</p>
  </Modal>
);
}

```

```

export function Stat({ label, value, hint, tone }) {
  return (
    <div className={cx('stat', tone && `stat-${tone}`)}>
      <div className="stat-label">{label}</div>
      <div className="stat-value">{value}</div>
      {hint && <div className="stat-hint">{hint}</div>}
    </div>
  );
}

```

```
// ===== Файл: server/routes/admin.js =====
```

```

import express from 'express';
import fs from 'node:fs';
import path from 'node:path';
import os from 'node:os';
import { hashPassword, requirePerm } from '../auth.js';
import { getSettings, saveSettings } from '../settings.js';
import { audit, replan } from '../services/orders.js';
import { bad, notFound } from '../util.js';

```

```

const wrap = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
const ROLES = ['admin', 'operator', 'master', 'mechanic'];

```

```

export default function adminRoutes(db) {
  const r = express.Router();

  r.get('/settings', (_req, res) => res.json(getSettings(db)));

  r.put('/settings', requirePerm('settings.write'), (req, res) => {
    const b = req.body || {};
    const re = /^d{2}:d{2}$/;
    for (const k of ['work_start', 'work_end', 'lunch_start', 'lunch_end']) if (b[k] && !re.test(b[k])) throw bad(`Время в формате ЧЧ:ММ (${k})`);
    if (b.work_start && b.work_end && b.work_start >= b.work_end) throw bad('Начало рабочего дня должно быть раньше конца');
    if (b.work_days && (!Array.isArray(b.work_days) || !b.work_days.length)) throw bad('Выберите рабочие дни');
    const s = saveSettings(db, b);
    audit(db, req.user, 'settings', null, 'update', JSON.stringify(b));
    replan(db);
    res.json(s);
  });
}

```

```

});

r.get('/users', requirePerm('users.write'), (_req, res) => {
  res.json(db.prepare(`SELECT u.id, u.login, u.name, u.role, u.employee_id, u.active, u.create
d_at, e.name AS employee_name
  FROM users u LEFT JOIN employees e ON e.id = u.employee_id ORDER BY u.active DESC, u.role,
  u.login`).all());
});

r.post('/users', requirePerm('users.write'), (req, res) => {
  const b = req.body || {};
  const login = String(b.login || '').trim().toLowerCase();
  if (! /^[a-z0-9._-]{3,32}$/.test(login)) throw bad('Логин: 3–32 символа, латиница, цифры, . _
-');
  if (!ROLES.includes(b.role)) throw bad('Неверная роль');
  if (!b.password || String(b.password).length < 6) throw bad('Пароль – не короче 6 символов')
;
  const id = db.prepare('INSERT INTO users(login, name, pass_hash, role, employee_id) VALUES (
?, ?, ?, ?, ?)')
    .run(login, String(b.name || login).trim(), hashPassword(b.password), b.role, b.employee_i
d || null).lastInsertRowid;
  audit(db, req.user, 'user', id, 'create', login);
  res.status(201).json({ id });
});

r.put('/users/:id', requirePerm('users.write'), (req, res) => {
  const u = db.prepare('SELECT * FROM users WHERE id = ?').get(req.params.id);
  if (!u) throw notFound('Пользователь');
  const b = req.body || {};
  if (b.role && !ROLES.includes(b.role)) throw bad('Неверная роль');
  if (u.id === req.user.id && (b.role && b.role !== 'admin' || b.active === false)) throw bad(
'Нельзя лишить прав самого себя');
  db.prepare('UPDATE users SET name = ?, role = ?, employee_id = ?, active = ? WHERE id = ?').
run(
  b.name ?? u.name, b.role ?? u.role, b.employee_id !== undefined ? b.employee_id || null :
u.employee_id,
  b.active !== undefined ? (b.active ? 1 : 0) : u.active, u.id,
);
  if (b.password) {
    if (String(b.password).length < 6) throw bad('Пароль – не короче 6 символов');
    db.prepare('UPDATE users SET pass_hash = ? WHERE id = ?').run(hashPassword(b.password), u.
id);
    db.prepare('DELETE FROM sessions WHERE user_id = ?').run(u.id);
  }
  if (b.active === false) db.prepare('DELETE FROM sessions WHERE user_id = ?').run(u.id);
  audit(db, req.user, 'user', u.id, 'update', JSON.stringify({ ...b, password: b.password ? '*'
:**' : undefined }));
  res.json({ ok: true });
});

```

```

    r.get('/audit', requirePerm('users.write'), (req, res) => {
      res.json(db.prepare(`SELECT a.*, u.name AS user_name FROM audit_log a LEFT JOIN users u ON u
        .id = a.user_id
        ORDER BY a.id DESC LIMIT ?`).all(Math.min(Number(req.query.limit) || 200, 2000)));
    });

    // Резервная копия базы данных (консистентный снимок)
    r.get('/admin/backup', requirePerm('settings.write'), wrap(async (_req, res) => {
      const file = path.join(os.tmpdir(), `fiberfleet-backup-${Date.now()}.db`);
      await db.backup(file);
      res.download(file, `fiberfleet-${new Date().toISOString().slice(0, 10)}.db`, () => fs.rm(file, { force: true }, () => {}));
    }));

    return r;
  }

  // ===== Файл: server/routes/auth.js =====
  import express from 'express';
  import { authMiddleware, createSession, hashPassword, PERMISSIONS, verifyPassword } from '../auth.js';
  import { bad, HttpError } from '../util.js';

  export default function authRoutes(db) {
    const r = express.Router();

    r.post('/login', (req, res) => {
      const { login, password } = req.body || {};
      const u = db.prepare('SELECT * FROM users WHERE login = ? AND active = 1').get(String(login || '').trim().toLowerCase());
      if (!u || !verifyPassword(password || '', u.pass_hash)) throw new HttpError(401, 'Неверный логин или пароль');
      const token = createSession(db, u.id);
      db.prepare("DELETE FROM sessions WHERE expires_at < ?").run(new Date().toISOString());
      res.json({ token, user: publicUser(u) });
    });

    r.use(authMiddleware(db));

    r.get('/me', (req, res) => res.json({ user: publicUser(req.user) }));

    r.post('/logout', (req, res) => {
      db.prepare('DELETE FROM sessions WHERE token = ?').run(req.token);
      res.json({ ok: true });
    });

    r.post('/password', (req, res) => {
      const { current, next } = req.body || {};
      const u = db.prepare('SELECT * FROM users WHERE id = ?').get(req.user.id);

```

```

    if (!verifyPassword(current || '', u.pass_hash)) throw bad('Текущий пароль указан неверно');
    if (!next || String(next).length < 6) throw bad('Новый пароль – не короче 6 символов');
    db.prepare('UPDATE users SET pass_hash = ? WHERE id = ?').run(hashPassword(next), u.id);
    db.prepare('DELETE FROM sessions WHERE user_id = ? AND token <> ?').run(u.id, req.token);
    res.json({ ok: true });
  });

  return r;
}

export function publicUser(u) {
  return {
    id: u.id, login: u.login, name: u.name, role: u.role, employee_id: u.employee_id ?? null,
    permissions: PERMISSIONS[u.role] || [],
  };
}

// ===== Файл: server/routes/maintenance.js =====
import express from 'express';
import { requirePerm } from '../auth.js';
import { fleetMaintenance } from '../domain/maintenance.js';
import { bad, notFound, num } from '../util.js';

export default function maintenanceRoutes(db) {
  const r = express.Router();

  r.get('/maintenance', (req, res) => {
    let rows = fleetMaintenance(db);
    if (req.query.status) rows = rows.filter((x) => x.status === req.query.status);
    res.json(rows);
  });

  r.get('/maintenance/rules', (_req, res) => {
    res.json(db.prepare(`SELECT m.*, w.name AS work_name FROM maintenance_rules m LEFT JOIN work
_types w ON w.id = m.work_type_id
ORDER BY m.vehicle_type, m.key, m.brand_match`).all()));
  });

  const fields = (b) => ({
    key: String(b.key || '').trim() || 'custom',
    name: String(b.name || '').trim(),
    vehicle_type: b.vehicle_type === 'trailer' ? 'trailer' : 'tractor',
    brand_match: String(b.brand_match || '').trim() || null,
    interval_km: num(b.interval_km),
    interval_days: num(b.interval_days),
    warn_km: num(b.warn_km) ?? 5000,
    warn_days: num(b.warn_days) ?? 14,
    work_type_id: num(b.work_type_id),
    active: b.active === false || b.active === 0 ? 0 : 1,
  });

```

```

});

const validate = (f) => {
  if (!f.name) throw bad('Укажите название правила');
  if (!f.interval_km && !f.interval_days) throw bad('Укажите интервал в км и/или в днях');
};

r.post('/maintenance/rules', requirePerm('maintenance.write'), (req, res) => {
  const f = fields(req.body || {});
  validate(f);
  const id = db.prepare(`INSERT INTO maintenance_rules(key, name, vehicle_type, brand_match, interval_km, interval_days, warn_km, warn_days, work_type_id, active)
    VALUES (@key, @name, @vehicle_type, @brand_match, @interval_km, @interval_days, @warn_km, @warn_days, @work_type_id, @active)`).run(f).lastInsertRowid;
  res.status(201).json({ id });
});

r.put('/maintenance/rules/:id', requirePerm('maintenance.write'), (req, res) => {
  const cur = db.prepare('SELECT id FROM maintenance_rules WHERE id = ?').get(req.params.id);
  if (!cur) throw notFound('Правило');
  const f = fields(req.body || {});
  validate(f);
  db.prepare(`UPDATE maintenance_rules SET key=@key, name=@name, vehicle_type=@vehicle_type, brand_match=@brand_match, interval_km=@interval_km,
    interval_days=@interval_days, warn_km=@warn_km, warn_days=@warn_days, work_type_id=@work_type_id, active=@active WHERE id=@id`).run({ ...f, id: cur.id });
  res.json({ ok: true });
});

r.delete('/maintenance/rules/:id', requirePerm('maintenance.write'), (req, res) => {
  db.prepare('DELETE FROM maintenance_rules WHERE id = ?').run(req.params.id);
  res.json({ ok: true });
});

// Отметить, что ТО выполнено ранее (до ведения учёта в системе)
r.post('/maintenance/baseline', requirePerm('maintenance.write'), (req, res) => {
  const b = req.body || {};
  const rule = db.prepare('SELECT * FROM maintenance_rules WHERE id = ?').get(b.rule_id);
  if (!rule) throw notFound('Правило');
  const m = num(b.done_mileage);
  if (m == null && !b.done_at) throw bad('Укажите пробег и/или дату выполнения');
  db.prepare(`INSERT INTO maintenance_baseline(rule_id, vehicle_type, vehicle_id, done_mileage, done_at) VALUES (?, ?, ?, ?, ?)
    ON CONFLICT(rule_id, vehicle_type, vehicle_id) DO UPDATE SET done_mileage = excluded.done_mileage, done_at = excluded.done_at`)
    .run(rule.id, rule.vehicle_type, Number(b.vehicle_id), m, b.done_at || new Date().toISOString());
  res.json({ ok: true });
});

```

```

    return r;
}

// ===== Файл: server/routes/orders.js =====
import express from 'express';
import { can, requirePerm } from '../auth.js';
import { nowIso } from '../db.js';
import { publicTemplate, PRIORITIES } from '../domain/checklist.js';
import { factorFor, loadEmployees, speedStats } from '../domain/scheduler.js';
import { recordMileage } from '../domain/maintenance.js';
import { calendarFor } from '../settings.js';
import { audit, createOrder, loadOrder, previewPlan, refreshOrderState, replan } from '../services/orders.js';
import { bad, HttpError, notFound, num } from '../util.js';

const ACTIVE = ['planned', 'plan_error', 'approved', 'in_progress'];

export default function orderRoutes(db) {
    const r = express.Router();

    r.get('/checklist', (_req, res) => res.json(publicTemplate()));

    r.get('/orders', (req, res) => {
        const where = [];
        const p = {};
        const st = req.query.status || 'active';
        if (st === 'active') where.push('o.status IN (${ACTIVE.map((s) => `'$${s}'`).join(',')})`);
        else if (st !== 'all') { where.push('o.status = @status'); p.status = st; }
        if (req.query.q) {
            where.push('(o.number LIKE @q OR tr.plate LIKE @q OR tl.plate LIKE @q)');
            p.q = `_${String(req.query.q).toUpperCase().replace(/\s+/g, '')}_`;
        }
        if (req.query.from) { where.push('o.inspected_at >= @from'); p.from = req.query.from; }
        if (req.query.to) { where.push('o.inspected_at < @to'); p.to = req.query.to; }
        if (req.query.vehicle_type && req.query.vehicle_id) {
            where.push(req.query.vehicle_type === 'tractor' ? 'o.tractor_id = @vid' : 'o.trailer_id = @vid');
            p.vid = Number(req.query.vehicle_id);
        }
        const rows = db.prepare(`SELECT o.id, o.number, o.status, o.inspected_at, o.release_deadline
, o.closed_at, o.plan_errors,
    o.tractor_id, o.trailer_id, tr.plate AS tractor_plate, tr.brand AS tractor_brand, tr.model AS tractor_model,
    tl.plate AS trailer_plate, tl.model AS trailer_model,
    (SELECT COUNT(*) FROM tasks t WHERE t.order_id = o.id AND t.status <> 'cancelled') AS tasks_total,
    (SELECT COUNT(*) FROM tasks t WHERE t.order_id = o.id AND t.status = 'done') AS tasks_done,
    (SELECT MIN(priority) FROM tasks t WHERE t.order_id = o.id AND t.status NOT IN ('done', '

```

```

cancelled')) AS top_priority,
    (SELECT MAX(s.end_at) FROM schedule s JOIN tasks t ON t.id = s.task_id WHERE t.order_id
= o.id AND t.status <> 'cancelled') AS planned_end,
    (SELECT COALESCE(SUM(price),0) FROM tasks t WHERE t.order_id = o.id AND t.status <> 'can
celled') AS total_price
FROM repair_orders o LEFT JOIN tractors tr ON tr.id = o.tractor_id LEFT JOIN trailers tl O
N tl.id = o.trailer_id
    ${where.length ? 'WHERE ' + where.join(' AND ') : ''}
ORDER BY o.inspected_at DESC LIMIT 500`).all(p);
for (const o of rows) o.error_count = JSON.parse(o.plan_errors || '[]').length, delete o.pla
n_errors;
res.json(rows);
});

r.post('/orders/preview', requirePerm('orders.write'), (req, res) => {
    res.json(previewPlan(db, req.body || {}));
});

r.post('/orders', requirePerm('orders.write'), (req, res) => {
    const b = req.body || {};
    if (!b.tractor_id && !b.trailer_id) throw bad('Выберите тягач и/или прицеп');
    if (b.release_deadline && b.release_deadline < nowIso()) throw bad('Дата выпуска с ремонта у
же прошла');
    const id = createOrder(db, {
        ...b,
        tractor_mileage: num(b.tractor_mileage), trailer_mileage: num(b.trailer_mileage), reefer_h
ours: num(b.reefer_hours),
    }, req.user);
    res.status(201).json(loadOrder(db, id));
});

r.get('/orders/:id', (req, res) => res.json(loadOrder(db, req.params.id)));

r.put('/orders/:id', requirePerm('orders.write'), (req, res) => {
    const o = db.prepare('SELECT * FROM repair_orders WHERE id = ?').get(req.params.id);
    if (!o) throw notFound('Ремонт');
    const b = req.body || {};
    const tx = db.transaction() => {
        if (b.notes !== undefined) db.prepare('UPDATE repair_orders SET notes = ? WHERE id = ?').r
un(b.notes || null, o.id);
        if (b.release_deadline !== undefined && b.release_deadline !== o.release_deadline) {
            db.prepare('UPDATE repair_orders SET release_deadline = ? WHERE id = ?').run(b.release_d
eadline || null, o.id);
            // пересчитать автоматические сроки
            const cal = calendarFor(db);
            for (const t of db.prepare(`SELECT id, priority FROM tasks WHERE order_id = ? AND deadli
ne_manual = 0 AND status NOT IN ('done','cancelled')`).all(o.id)) {
                const dl = b.release_deadline || cal.addWorkDays(o.inspected_at, PRIORITIES[t.priority
] ?? 5);
                db.prepare('UPDATE tasks SET deadline = ? WHERE id = ?').run(dl, t.id);
            }
        }
    };
    tx.run();
    res.json(loadOrder(db, req.params.id));
});

```

```

    }
  }
  for (const [k, type] of [['tractor_mileage', 'tractor'], ['trailer_mileage', 'trailer']])
  {
    if (b[k] === undefined) continue;
    const m = num(b[k]);
    db.prepare(`UPDATE repair_orders SET ${k} = ? WHERE id = ?`).run(m, o.id);
    db.prepare(`UPDATE tasks SET mileage = ? WHERE order_id = ? AND vehicle_type = ? AND sta
tus <> 'done'`).run(m, o.id, type);
    const vid = type === 'tractor' ? o.tractor_id : o.trailer_id;
    if (m && vid) recordMileage(db, { type, id: vid, mileage: m, source: 'repair', refId: o.
id, userId: req.user.id });
  }
});
tx();
audit(db, req.user, 'order', o.id, 'update', JSON.stringify(b));
replan(db, o.id);
res.json(loadOrder(db, o.id));
});

r.post('/orders/:id/replan', requirePerm('plan.write'), (req, res) => {
  loadOrder(db, req.params.id);
  replan(db, Number(req.params.id));
  res.json(loadOrder(db, req.params.id));
});

r.post('/orders/:id/approve', requirePerm('plan.write'), (req, res) => {
  const id = Number(req.params.id);
  replan(db, id);
  const o = loadOrder(db, id);
  if (o.plan_errors.length) {
    throw new HttpError(409, 'План нельзя утвердить: есть ошибки планирования', o.plan_errors)
  }
  if (['planned', 'plan_error'].includes(o.status)) {
    db.prepare("UPDATE repair_orders SET status = 'approved', updated_at = ? WHERE id = ?").ru
n(nowIso(), id);
  }
  audit(db, req.user, 'order', id, 'approve');
  res.json(loadOrder(db, id));
});

r.post('/orders/:id/cancel', requirePerm('orders.write'), (req, res) => {
  const id = Number(req.params.id);
  loadOrder(db, id);
  db.transaction(() => {
    db.prepare(`DELETE FROM schedule WHERE task_id IN (SELECT id FROM tasks WHERE order_id = ?
AND status <> 'done')`).run(id);
    db.prepare(`UPDATE tasks SET status = 'cancelled', updated_at = ? WHERE order_id = ? AND s
tatus NOT IN ('done')`).run(nowIso(), id);
  });
});

```

```

    db.prepare(`UPDATE repair_orders SET status = 'cancelled', closed_at = ?, updated_at = ? W
HERE id = ?`).run(nowIso(), nowIso(), id);
  })();
  audit(db, req.user, 'order', id, 'cancel');
  replan(db);
  res.json(loadOrder(db, id));
});

r.delete('/orders/:id', requirePerm('orders.delete'), (req, res) => {
  const id = Number(req.params.id);
  db.prepare('DELETE FROM repair_orders WHERE id = ?').run(id);
  audit(db, req.user, 'order', id, 'delete');
  replan(db);
  res.json({ ok: true });
});

// ---- Работы (задачи) ----

r.post('/orders/:id/tasks', requirePerm('plan.write'), (req, res) => {
  const o = db.prepare('SELECT * FROM repair_orders WHERE id = ?').get(req.params.id);
  if (!o) throw notFound('Ремонт');
  if (['done', 'cancelled'].includes(o.status)) throw bad('Ремонт закрыт – добавление работ не
возможно');
  const b = req.body || {};
  const wt = db.prepare('SELECT * FROM work_types WHERE id = ?').get(b.work_type_id);
  if (!wt) throw bad('Выберите работу из справочника');
  const vt = b.vehicle_type === 'trailer' ? 'trailer' : 'tractor';
  const vid = vt === 'tractor' ? o.tractor_id : o.trailer_id;
  if (!vid) throw bad(vt === 'tractor' ? 'В ремонте нет тягача' : 'В ремонте нет прицепа');
  const priority = Math.min(4, Math.max(1, Number(b.priority) || 2));
  const cal = calendarFor(db);
  const deadline = b.deadline || o.release_deadline || cal.addWorkDays(nowIso(), PRIORITIES[pri
ority].days);
  const id = db.prepare(`INSERT INTO tasks(order_id, vehicle_type, vehicle_id, work_type_id, d
escription, location, reason, source, seq,
    priority, priority_manual, deadline, deadline_manual, hours, price, salary, mileage, sta
tus)
    VALUES (?, ?, ?, ?, ?, ?, ?, 'manual', ?, ?, 1, ?, ?, ?, ?, ?, 'planned')`).run(
    o.id, vt, vid, wt.id, wt.name, b.location || null, b.comment ? `Добавлено вручную: ${b.com
ment}` : 'Добавлено вручную',
    /^(диагност|осмотр|тест)/i.test(wt.name) ? 1 : 2, priority, deadline, b.deadline ? 1 : 0,
    num(b.hours) ?? wt.hours, wt.price, wt.salary, vt === 'tractor' ? o.tractor_mileage : o.tr
ailer_mileage,
  ).lastInsertRowid;
  if (o.status === 'approved') db.prepare("UPDATE repair_orders SET status = 'planned' WHERE i
d = ?").run(o.id);
  audit(db, req.user, 'task', id, 'create', wt.name);
  replan(db, o.id);
  res.status(201).json(loadOrder(db, o.id));
});

```

```

r.put('/tasks/:id', (req, res) => {
  const t = db.prepare('SELECT * FROM tasks WHERE id = ?').get(req.params.id);
  if (!t) throw notFound('Работа');
  const b = req.body || {};
  const planFields = ['priority', 'deadline', 'hours', 'description', 'location', 'comment', '
price', 'salary', 'seq'];
  const reassign = b.employee_id !== undefined || b.start_at !== undefined || b.unlock !== und
efined;
  if (planFields.some((f) => b[f] !== undefined) && !can(req.user, 'plan.write')) throw new Ht
tpError(403, 'Недостаточно прав для корректировки плана');
  if (reassign && !can(req.user, 'tasks.reassign')) throw new HttpError(403, 'Переназначать за
дачи может только администратор');
  if (['done', 'cancelled'].includes(t.status) && planFields.some((f) => ['priority', 'deadlin
e', 'hours'].includes(f) && b[f] !== undefined)) {
    throw bad('Работа уже закрыта');
  }

  const tx = db.transaction(() => {
    const set = {};
    if (b.priority !== undefined) {
      const p = Number(b.priority);
      if (!(p >= 1 && p <= 4)) throw bad('Приоритет: от 1 до 4');
      set.priority = p; set.priority_manual = 1;
    }
    if (b.deadline !== undefined) { set.deadline = b.deadline || null; set.deadline_manual = 1
; }
    if (b.hours !== undefined) { const h = num(b.hours); if (!(h > 0)) throw bad('Норма времен
и должна быть больше 0'); set.hours = h; }
    for (const f of ['description', 'location', 'comment']) if (b[f] !== undefined) set[f] = b
[f] || null;
    for (const f of ['price', 'salary']) if (b[f] !== undefined) set[f] = num(b[f]);
    if (b.seq !== undefined) set.seq = Math.max(0, Math.min(3, Number(b.seq) || 0));
    if (set.description === null) throw bad('Описание работы не может быть пустым');

    if (b.employee_id !== undefined) {
      if (b.employee_id) {
        const e = db.prepare('SELECT id FROM employees WHERE id = ? AND active = 1').get(b.emp
loyee_id);
        if (!e) throw bad('Сотрудник не найден');
        set.employee_id = e.id; set.assigned_manual = 1;
      } else {
        set.assigned_manual = 0;
        db.prepare('UPDATE schedule SET locked = 0 WHERE task_id = ?').run(t.id);
      }
    }
    if (Object.keys(set).length) {
      db.prepare(`UPDATE tasks SET ${Object.keys(set).map((k) => `${k} = @${k}`)}.join(', ')`,
updated_at = @ts WHERE id = @id`)
        .run({ ...set, ts: nowIso(), id: t.id });
    }
  });

```

```

    }
    // ручная установка времени начала – фиксирует задачу в расписании
    if (b.start_at) {
        const cur = db.prepare('SELECT * FROM tasks WHERE id = ?').get(t.id);
        if (!cur.employee_id) throw bad('Сначала выберите исполнителя');
        const cal = calendarFor(db);
        const { emps } = loadEmployees(db);
        const emp = emps.find((e) => e.id === cur.employee_id);
        const wt = cur.work_type_id ? db.prepare('SELECT category FROM work_types WHERE id = ?')
        .get(cur.work_type_id) : null;
        const dur = Math.max(5, Math.ceil(((cur.hours || 1) * 60 * factorFor(emp || {}, wt?.category, speedStats(db))) / 5) * 5);
        const s = cal.toWork(b.start_at);
        const start = cal.fromWork(s), end = cal.fromWork(s + dur, true);
        const overlap = db.prepare('SELECT t.description FROM schedule s JOIN tasks t ON t.id = s.task_id
        WHERE s.employee_id = ? AND s.task_id <> ? AND t.status IN ('planned','in_progress','paused')
        AND s.start_at < ? AND s.end_at > ? AND (s.locked = 1 OR t.status <> 'planned')').get(
        cur.employee_id, t.id, end, start);
        if (overlap) throw bad('Сотрудник в это время занят: «${overlap.description}»');
        db.prepare('INSERT INTO schedule(task_id, employee_id, vehicle_type, vehicle_id, start_at, end_at, locked)
        VALUES (?, ?, ?, ?, ?, ?, 1) ON CONFLICT(task_id) DO UPDATE SET employee_id = excluded.employee_id,
        start_at = excluded.start_at, end_at = excluded.end_at, locked = 1')
        .run(t.id, cur.employee_id, cur.vehicle_type, cur.vehicle_id, start, end);
    } else if (b.unlock) {
        db.prepare('UPDATE schedule SET locked = 0 WHERE task_id = ?').run(t.id);
    }
    });
    tx();
    audit(db, req.user, 'task', t.id, 'update', JSON.stringify(b));
    if (t.order_id) {
        const o = db.prepare('SELECT status FROM repair_orders WHERE id = ?').get(t.order_id);
        if (o?.status === 'approved' && !reassign) db.prepare("UPDATE repair_orders SET status = 'planned' WHERE id = ?").run(t.order_id);
    }
    // ручное назначение влияет на загрузку – пересчитываем всех
    replan(db, reassign ? null : t.order_id);
    res.json(t.order_id ? loadOrder(db, t.order_id) : { ok: true });
});

r.post('/tasks/:id/status', requirePerm('tasks.status'), (req, res) => {
    const t = db.prepare('SELECT * FROM tasks WHERE id = ?').get(req.params.id);
    if (!t) throw notFound('Работа');
    const { status, comment } = req.body || {};
    const allowed = { planned: ['in_progress', 'cancelled', 'done'], in_progress: ['paused', 'done', 'cancelled', 'planned'], paused: ['in_progress', 'done', 'cancelled'], done: ['in_progress', 'cancelled', 'planned'] };

```

```

    if (!allowed[t.status]?.includes(status)) throw bad(`Нельзя перевести работу из «${t.status}
» в «${status}»`);
    const ts = req.body.at || nowIso();
    const cal = calendarFor(db);
    db.transaction(() => {
        const upd = { status, ts: nowIso(), id: t.id, comment: comment ?? t.comment };
        let extra = '';
        if (status === 'in_progress' && !t.started_at) { extra += ', started_at = @started_at'; up
d.started_at = ts; }
        if (status === 'done') {
            const started = t.started_at || db.prepare('SELECT start_at FROM schedule WHERE task_id
= ?').get(t.id)?.start_at || ts;
            let actual = num(req.body.actual_hours);
            if (actual == null) actual = Math.max(0.1, Math.round(((cal.toWork(ts) - cal.toWork(star
ted)) / 60) * 100) / 100);
            if (!t.employee_id && !req.body.employee_id) throw bad('Укажите исполнителя работы');
            extra += ', finished_at = @finished_at, actual_hours = @actual_hours, started_at = COALE
SCE(started_at, @started_at)';
            Object.assign(upd, { finished_at: ts, actual_hours: actual, started_at: started });
            if (req.body.employee_id) { extra += ', employee_id = @employee_id'; upd.employee_id = N
umber(req.body.employee_id); }
            const m = num(req.body.mileage);
            if (m != null) { extra += ', mileage = @mileage'; upd.mileage = m; }
            // факт в расписании
            db.prepare('UPDATE schedule SET start_at = ?, end_at = ?, locked = 1 WHERE task_id = ?')
.run(started, ts, t.id);
            if (m != null) recordMileage(db, { type: t.vehicle_type, id: t.vehicle_id, mileage: m, a
t: ts, source: 'repair', refId: t.order_id, userId: req.user.id });
        }
        if (status === 'in_progress' && t.status === 'done') { extra += ', finished_at = NULL, act
ual_hours = NULL'; }
        if (status === 'cancelled') db.prepare('DELETE FROM schedule WHERE task_id = ?').run(t.id)
;
        if (status === 'in_progress') {
            // работа началась: зафиксировать начало в расписании
            const s = db.prepare('SELECT * FROM schedule WHERE task_id = ?').get(t.id);
            if (s) {
                const len = Math.max(5, cal.toWork(s.end_at) - cal.toWork(s.start_at));
                const st = t.started_at || ts;
                db.prepare('UPDATE schedule SET start_at = ?, end_at = ?, locked = 1 WHERE task_id = ?
')
                .run(st, cal.fromWork(cal.toWork(st) + len, true), t.id);
            } else if (t.employee_id) {
                const st = t.started_at || ts;
                db.prepare('INSERT INTO schedule(task_id, employee_id, vehicle_type, vehicle_id, start
_at, end_at, locked) VALUES (?, ?, ?, ?, ?, ?, 1)')
                .run(t.id, t.employee_id, t.vehicle_type, t.vehicle_id, st, cal.fromWork(cal.toWork(
st) + Math.ceil((t.hours || 1) * 60), true));
            }
        }
    }
}

```

```

    if (status === 'planned') db.prepare('UPDATE schedule SET locked = 0 WHERE task_id = ?').run(t.id);
    db.prepare(`UPDATE tasks SET status = @status, comment = @comment, updated_at = @ts${extra} WHERE id = @id`).run(upd);
  })();
  audit(db, req.user, 'task', t.id, 'status', `${t.status} → ${status}`);
  replan(db);
  if (t.order_id) refreshOrderState(db, t.order_id);
  res.json(t.order_id ? loadOrder(db, t.order_id) : { ok: true });
});

r.delete('/tasks/:id', requirePerm('plan.write'), (req, res) => {
  const t = db.prepare('SELECT * FROM tasks WHERE id = ?').get(req.params.id);
  if (!t) throw notFound('Работа');
  if (t.status === 'done') throw bad('Выполненную работу удалить нельзя');
  if (t.status === 'in_progress') throw bad('Работа выполняется – сначала остановите её или отмените');
  db.prepare('DELETE FROM tasks WHERE id = ?').run(t.id);
  audit(db, req.user, 'task', t.id, 'delete', t.description);
  replan(db, t.order_id);
  res.json(t.order_id ? loadOrder(db, t.order_id) : { ok: true });
});

// ---- Установленные запчасти ----

r.post('/orders/:id/parts', requirePerm('parts.write'), (req, res) => {
  const o = db.prepare('SELECT * FROM repair_orders WHERE id = ?').get(req.params.id);
  if (!o) throw notFound('Ремонт');
  const b = req.body || {};
  let task = null;
  if (b.task_id) {
    task = db.prepare('SELECT * FROM tasks WHERE id = ? AND order_id = ?').get(b.task_id, o.id);
  }
  if (!task) throw bad('Работа не найдена в этом ремонте');
  }
  let part = b.part_id ? db.prepare('SELECT * FROM parts WHERE id = ?').get(b.part_id) : null;
  const name = (part?.name || b.name || '').trim();
  if (!name) throw bad('Укажите запчасть');
  const qty = num(b.qty) ?? 1;
  if (!(qty > 0)) throw bad('Количество должно быть больше 0');
  if (!part && b.save_to_catalog) {
    const pid = db.prepare(`INSERT INTO parts(code, article, name, unit, price, source) VALUES(?, ?, ?, ?, ?, 'manual')`).run(b.code || null, b.article || null, name, b.unit || 'шт', num(b.price)).lastInsertRowid;
    part = db.prepare('SELECT * FROM parts WHERE id = ?').get(pid);
  }
  const vt = task?.vehicle_type || (b.vehicle_type === 'trailer' ? 'trailer' : (o.tractor_id ? 'tractor' : 'trailer'));
  const vid = task?.vehicle_id || (vt === 'tractor' ? o.tractor_id : o.trailer_id);

```

```

    if (!vid) throw bad('Не определено ТС для установки');
    db.prepare(`INSERT INTO installed_parts(task_id, order_id, vehicle_type, vehicle_id, part_id
, code, article, name, unit, qty, price, source, mileage, installed_at, user_id)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`).run(
    task?.id ?? null, o.id, vt, vid, part?.id ?? null, part?.code ?? b.code ?? null, part?.art
icle ?? b.article ?? null,
    name, part?.unit || b.unit || 'шт', qty, num(b.price) ?? part?.price ?? null, part?.source
|| 'manual',
    vt === 'tractor' ? o.tractor_mileage : o.trailer_mileage, b.installed_at || nowIso(), req.
user.id,
    );
    audit(db, req.user, 'installed_part', o.id, 'create', name);
    res.status(201).json(loadOrder(db, o.id));
  });

  r.delete('/installed-parts/:id', requirePerm('parts.write'), (req, res) => {
    const p = db.prepare('SELECT * FROM installed_parts WHERE id = ?').get(req.params.id);
    if (!p) throw notFound('Запчасть');
    if (p.exported_at) throw bad('Запчасть уже выгружена в 1С – удаление запрещено');
    db.prepare('DELETE FROM installed_parts WHERE id = ?').run(p.id);
    res.json(p.order_id ? loadOrder(db, p.order_id) : { ok: true });
  });

  return r;
}

// ===== Файл: server/routes/parts.js =====
import express from 'express';
import multer from 'multer';
import ExcelJS from 'exceljs';
import { requirePerm } from '../auth.js';
import { nowIso } from '../db.js';
import { getSettings } from '../settings.js';
import { audit } from '../services/orders.js';
import { bad, fmtDateTime, notFound, num } from '../util.js';

const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 20 * 1024 * 1024 }
});
const wrap = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

// ---- Разбор файлов номенклатуры из 1С ----

const HEADERS = {
  code: ['код', 'код 1с', 'ид', 'id', 'code'],
  article: ['артикул', 'article', 'арт', 'арт.'],
  name: ['наименование', 'номенклатура', 'название', 'name', 'товар'],
  unit: ['ед', 'ед.', 'ед. изм.', 'ед.изм.', 'единица', 'единица измерения', 'unit'],
  price: ['цена', 'цена с ндс', 'цена продажи', 'price', 'стоимость'],
};

```

```
function mapHeader(cells) {
  const idx = {};
  cells.forEach((c, i) => {
    const h = String(c ?? '').trim().toLowerCase();
    for (const [k, names] of Object.entries(HEADERS)) if (idx[k] == null && names.includes(h)) i
    dx[k] = i;
  });
  return idx;
}
```

```
function rowsToParts(rows) {
  let hi = rows.findIndex((r) => { const m = mapHeader(r); return m.name != null; });
  if (hi < 0) throw bad('Не найдена строка заголовков: нужна как минимум колонка «Наименование»
(также: Код, Артикул, Ед. изм., Цена)');
  const m = mapHeader(rows[hi]);
  const out = [];
  for (const r of rows.slice(hi + 1)) {
    const name = String(r[m.name] ?? '').trim();
    if (!name) continue;
    out.push({
      code: m.code != null ? String(r[m.code] ?? '').trim() || null : null,
      article: m.article != null ? String(r[m.article] ?? '').trim() || null : null,
      name,
      unit: m.unit != null ? String(r[m.unit] ?? '').trim() || 'шт' : 'шт',
      price: m.price != null ? num(String(r[m.price] ?? '').replace(/\\s/g, '').replace(',', '.'))
    ) : null,
    });
  }
  return out;
}
```

```
function parseCsv(text) {
  const sep = (text.split('\\n')[0].match(/;/g) || []).length >= (text.split('\\n')[0].match(/,/g)
  || []).length ? ';' : ',';
  const rows = [];
  let row = [], cur = '', q = false;
  for (let i = 0; i < text.length; i++) {
    const ch = text[i];
    if (q) {
      if (ch === '"' && text[i + 1] === '"') { cur += '"'; i++; } else if (ch === '"') q = false
    ; else cur += ch;
    } else if (ch === '"') q = true;
    else if (ch === sep) { row.push(cur); cur = ''; }
    else if (ch === '\\n' || ch === '\\r') {
      if (ch === '\\r' && text[i + 1] === '\\n') i++;
      row.push(cur); rows.push(row); row = []; cur = '';
    } else cur += ch;
  }
  if (cur || row.length) { row.push(cur); rows.push(row); }
```

```

    return rows;
}

function decodeText(buf) {
    if (buf[0] === 0xef && buf[1] === 0xbb && buf[2] === 0xbf) return buf.slice(3).toString('utf8');
    const utf = buf.toString('utf8');
    if (!utf.includes('\0')) return utf;
    return new TextDecoder('windows-1251').decode(buf); // выгрузки 1С часто в cp1251
}

const xmlText = (s) => String(s || '').replace(/&lt;/g, '<').replace(/&gt;/g, '>').replace(/&quot;/g, '"').replace(/&apos;/g, "'").replace(/&amp;/g, '&').trim();
const tag = (block, name) => { const m = block.match(new RegExp(`<${name}?(?:\\s[>]*)?>([\\s\\S]*?)</${name}>`)); return m ? xmlText(m[1]) : null; };

// CommerceML 2 (import.xml / offers.xml) – стандартный обмен 1С
function parseCommerceML(xml) {
    const parts = [];
    for (const m of xml.matchAll(/<Товар>([\\s\\S]*?)</Товар>/g)) {
        const b = m[1];
        const unitM = b.match(/<БазоваяЕдиница[>]*?(?:НаименованиеПолное="([^"]*)"?)?[>]*([\\s\\S]*?)</БазоваяЕдиница>/);
        parts.push({ code: tag(b, 'Ид'), article: tag(b, 'Артикул'), name: tag(b, 'Наименование'), unit: unitM ? (xmlText(unitM[2]) || 'шт') : 'шт', price: null });
    }
    const prices = new Map();
    for (const m of xml.matchAll(/<Предложение>([\\s\\S]*?)</Предложение>/g)) {
        const id = tag(m[1], 'Ид');
        const price = num(tag(m[1], 'ЦенаЗаЕдиницу'));
        if (id && price != null) prices.set(id, price);
        if (id && !parts.some(p => p.code === id)) {
            parts.push({ code: id, article: tag(m[1], 'Артикул'), name: tag(m[1], 'Наименование'), unit: 'шт', price });
        }
    }
    for (const p of parts) if (prices.has(p.code)) p.price = prices.get(p.code);
    return parts.filter((p) => p.name || p.code);
}

function upsertParts(db, parts) {
    let added = 0, updated = 0;
    const findByCode = db.prepare('SELECT id FROM parts WHERE code = ?');
    const findByArt = db.prepare("SELECT id FROM parts WHERE code IS NULL AND article = ? AND name = ?");
    const ins = db.prepare(`INSERT INTO parts(code, article, name, unit, price, source) VALUES (@code, @article, @name, @unit, @price, '1c')`);
    const upd = db.prepare(`UPDATE parts SET article = COALESCE(@article, article), name = COALESCE(@name, name), unit = COALESCE(@unit, unit), price = COALESCE(@price, price), source = '1c', updated_at = @ts WHERE id = @id`);

```

```

db.transaction(() => {
  for (const p of parts) {
    const ex = (p.code && findByCode.get(p.code)) || (p.article && findByArt.get(p.article, p.name));
    if (ex) { upd.run({ ...p, id: ex.id, ts: nowIso() }); updated++; } else if (p.name) { ins.run(p); added++; }
  }
})();
return { added, updated, total: parts.length };
}

```

```
// ---- Выгрузка установленных запчастей для 1С ----
```

```

function installedQuery(db, { from, to, onlyNew }) {
  const where = [];
  const p = {};
  if (from) { where.push('ip.installed_at >= @from'); p.from = from; }
  if (to) { where.push('ip.installed_at < @to'); p.to = to; }
  if (onlyNew) where.push('ip.exported_at IS NULL');
  return db.prepare(`SELECT ip.*, o.number AS order_number, t.description AS work,
    CASE ip.vehicle_type WHEN 'tractor' THEN tr.plate ELSE tl.plate END AS plate
  FROM installed_parts ip LEFT JOIN repair_orders o ON o.id = ip.order_id LEFT JOIN tasks t ON t.id = ip.task_id
  LEFT JOIN tractors tr ON ip.vehicle_type = 'tractor' AND tr.id = ip.vehicle_id
  LEFT JOIN trailers tl ON ip.vehicle_type = 'trailer' AND tl.id = ip.vehicle_id
  ${where.length ? 'WHERE ' + where.join(' AND ') : ''} ORDER BY ip.installed_at`).all(p);
}

```

```

const esc = (s) => String(s ?? '').replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;');

```

```

export function installedToXml(rows, off) {
  const docs = new Map();
  for (const r of rows) {
    const k = r.order_number || 'без-ремонта';
    if (!docs.has(k)) docs.set(k, []);
    docs.get(k).push(r);
  }
  const lines = ['<?xml version="1.0" encoding="UTF-8"?>',
    `<ФайлОбмена ВерсияФормата="FiberFleet-1.0" ДатаФормирования="${new Date().toISOString()}" Программа="FiberFleet">`];
  for (const [num, items] of docs) {
    const first = items[0];
    lines.push(`<Документ Вид="СписаниеЗапчастейВРемонт" Номер="${esc(num)}" Дата="${esc(fmtDate(first.installed_at, off))}" TC="${esc(first.plate)}" ТипTC="${first.vehicle_type === 'tractor' ? 'Тягач' : 'Прицеп'}" Пробег="${first.mileage ?? ''}">`);
    for (const r of items) {
      lines.push(`<Строка Код="${esc(r.code)}" Артикул="${esc(r.article)}" Наименование="${esc(r.name)}" ЕдиницаИзмерения="${esc(r.unit)}" Количество="${r.qty}" Цена="${r.price ?? ''}" Сумма="${r.price != null ? Math.round(r.price * r.qty * 100) / 100 : ''}" Работа="${esc(r.work)}`);
    }
  }
}

```

```

    }" ДатаУстановки="${esc(fmtDateTime(r.installed_at, off))}" ИдЗаписи="${r.id}" />`;
    }
    lines.push(' </Документ>');
  }
  lines.push('</ФайлОбмена>');
  return lines.join('\n');
}

export default function partsRoutes(db) {
  const r = express.Router();

  r.get('/parts', (req, res) => {
    const q = `%${String(req.query.q || '').trim()}%`;
    res.json(db.prepare(`SELECT p.*, (SELECT COALESCE(SUM(qty),0) FROM installed_parts i WHERE i
    .part_id = p.id) AS installed_qty
    FROM parts p WHERE p.name LIKE @q OR p.code LIKE @q OR p.article LIKE @q ORDER BY p.name L
    IMIT @lim`)
    .all({ q, lim: Math.min(Number(req.query.limit) || 300, 2000) })));

  r.post('/parts', requirePerm('parts.write'), (req, res) => {
    const b = req.body || {};
    if (!b.name?.trim()) throw bad('Укажите наименование');
    const id = db.prepare(`INSERT INTO parts(code, article, name, unit, price, source) VALUES (?
    , ?, ?, ?, ?, ?)`
    .run(b.code?.trim() || null, b.article?.trim() || null, b.name.trim(), b.unit || 'шт', num
    (b.price), b.source === 'lc' ? 'lc' : 'manual').lastInsertRowid;
    res.status(201).json({ id });
  });

  r.put('/parts/:id', requirePerm('parts.write'), (req, res) => {
    const p = db.prepare('SELECT * FROM parts WHERE id = ?').get(req.params.id);
    if (!p) throw notFound('Запчасть');
    const b = req.body || {};
    db.prepare(`UPDATE parts SET code = ?, article = ?, name = ?, unit = ?, price = ?, updated_a
    t = ? WHERE id = ?`)
    .run(b.code?.trim() || null, b.article?.trim() || null, (b.name || p.name).trim(), b.unit
    || p.unit, num(b.price), nowIso(), p.id);
    res.json({ ok: true });
  });

  r.delete('/parts/:id', requirePerm('parts.write'), (req, res) => {
    db.prepare('DELETE FROM parts WHERE id = ?').run(req.params.id);
    res.json({ ok: true });
  });

  // Импорт номенклатуры из 1C: XLSX / CSV / CommerceML XML
  r.post('/parts/import', requirePerm('parts.write'), upload.single('file'), wrap(async (req, re
  s) => {
    if (!req.file) throw bad('Файл не получен');

```

```

    const name = req.file.originalname.toLowerCase();
    let parts;
    if (name.endsWith('.xlsx')) {
        const wb = new ExcelJS.Workbook();
        await wb.xlsx.load(req.file.buffer);
        const ws = wb.worksheets[0];
        const rows = [];
        ws.eachRow({ includeEmpty: false }, (row) => rows.push(row.values.slice(1).map((v) => (v &
& typeof v === 'object' ? v.result ?? v.text ?? '' : v))));
        parts = rowsToParts(rows);
    } else if (name.endsWith('.xml')) {
        parts = parseCommerceML(decodeText(req.file.buffer));
    } else if (name.endsWith('.csv') || name.endsWith('.txt')) {
        parts = rowsToParts(parseCsv(decodeText(req.file.buffer)));
    } else {
        throw bad('Поддерживаются файлы .xlsx, .csv, .xml (CommerceML)');
    }
    if (!parts.length) throw bad('В файле не найдено ни одной позиции');
    const result = upsertParts(db, parts);
    audit(db, req.user, 'parts', null, 'import', JSON.stringify(result));
    res.json(result);
}));

// HTTP-обмен с 1C: загрузка номенклатуры JSON-массивом [{code, article, name, unit, price}]
r.post('/onec/parts', requirePerm('parts.write'), (req, res) => {
    const list = Array.isArray(req.body) ? req.body : req.body?.items;
    if (!Array.isArray(list)) throw bad('Ожидается массив позиций');
    res.json(upsertParts(db, list.map((p) => ({
        code: p.code ? String(p.code) : null, article: p.article || null, name: p.name ? String(p.
name) : null, unit: p.unit || 'шт', price: num(p.price),
    })))));
});

r.get('/installed-parts', (req, res) => {
    res.json(installedQuery(db, { from: req.query.from, to: req.query.to, onlyNew: req.query.onl
y_new === '1' }));
});

// Выгрузка в 1C: xml | xlsx | csv | json; mark=1 – отметить выгруженные
r.get('/onec/export', requirePerm('parts.write'), wrap(async (req, res) => {
    const off = Number(getSettings(db).tz_offset_min ?? 180);
    const rows = installedQuery(db, { from: req.query.from, to: req.query.to, onlyNew: req.query
.only_new === '1' });
    const fmt = req.query.format || 'xml';
    if (req.query.mark === '1' && rows.length) {
        const ids = rows.map((r) => r.id);
        db.prepare(`UPDATE installed_parts SET exported_at = ? WHERE id IN (${ids.map(() => '?').j
oin(',')}')`).run(nowIso(), ...ids);
        audit(db, req.user, 'parts', null, 'export_1c', `${rows.length} строк, ${fmt}`);
    }
}

```

```

    const stamp = new Date().toISOString().slice(0, 10);
    if (fmt === 'json') return res.json(rows);
    if (fmt === 'xml') {
        res.setHeader('Content-Disposition', `attachment; filename="fiberfleet-1c-${stamp}.xml"`);
        return res.type('application/xml').send(installedToXml(rows, off));
    }
    const header = ['Дата', 'Документ', 'Тип ТС', 'Госномер', 'Код 1С', 'Артикул', 'Наименование',
        'Ед.', 'Количество', 'Цена', 'Сумма', 'Пробег', 'Работа'];
    const data = rows.map((r) => [fmtDateTime(r.installed_at, off), r.order_number, r.vehicle_type === 'tractor' ? 'Тягач' : 'Прицеп', r.plate, r.code, r.article, r.name, r.unit, r.qty, r.price, r.price != null ? r.price * r.qty : null, r.mileage, r.work]);
    if (fmt === 'csv') {
        const csv = [header, ...data].map((r) => r.map((v) => `${String(v ?? '').replace(/"/g, '"')}`)).join(';')).join('\r\n');
        res.setHeader('Content-Disposition', `attachment; filename="fiberfleet-1c-${stamp}.csv"`);
        return res.type('text/csv; charset=utf-8').send('' + csv);
    }
    const wb = new ExcelJS.Workbook();
    const ws = wb.addWorksheet('Установленные запчасти');
    ws.addRow(header).font = { bold: true };
    data.forEach((d) => ws.addRow(d));
    ws.columns.forEach((c, i) => { c.width = [17, 12, 9, 14, 12, 14, 40, 6, 10, 10, 12, 10, 40][i]; });
    res.setHeader('Content-Disposition', `attachment; filename="fiberfleet-1c-${stamp}.xlsx"`);
    res.type('application/vnd.openxmlformats-officedocument.spreadsheetml.sheet');
    await wb.xlsx.write(res);
    res.end();
}));

return r;
}

```

```
// ===== Файл: server/routes/reports.js =====
```

```

import express from 'express';
import { requirePerm } from '../auth.js';
import { getSettings } from '../settings.js';
import { buildReport, orderToPdf, reportToCsv, reportToPdf, reportToXlsx, REPORTS } from '../services/reports.js';
import { loadOrder } from '../services/orders.js';

```

```

const wrap = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
const disp = (name) => `attachment; filename="report.${name.split('.').pop()}"; filename*=UTF-8'${encodeURIComponent(name)}`;

```

```

export default function reportRoutes(db) {
    const r = express.Router();

    r.get('/reports', (_req, res) => res.json(REPORTS));
}

```

```

r.get('/reports/:type', requirePerm('reports.read'), wrap(async (req, res) => {
  const rep = buildReport(db, req.params.type, req.query);
  const company = getSettings(db).company || 'FiberFleet';
  const fmt = req.query.format || 'json';
  const stamp = new Date().toISOString().slice(0, 10);
  const base = `${rep.title} ${stamp}`;
  if (fmt === 'json') return res.json(rep);
  if (fmt === 'csv') {
    res.setHeader('Content-Disposition', disp(`${base}.csv`));
    return res.type('text/csv; charset=utf-8').send(reportToCsv(rep));
  }
  if (fmt === 'xlsx') {
    res.setHeader('Content-Disposition', disp(`${base}.xlsx`));
    res.type('application/vnd.openxmlformats-officedocument.spreadsheetml.sheet');
    await reportToXlsx(rep, res, company);
    return res.end();
  }
  if (fmt === 'pdf') {
    res.setHeader('Content-Disposition', disp(`${base}.pdf`));
    res.type('application/pdf');
    return reportToPdf(rep, res, company);
  }
  res.status(400).json({ error: 'Формат: json, xlsx, pdf, csv' });
}));

r.get('/orders/:id/pdf', (req, res) => {
  const order = loadOrder(db, req.params.id);
  const s = getSettings(db);
  res.setHeader('Content-Disposition', `${req.query.inline === '1' ? 'inline' : 'attachment'};
filename="order-${order.number}.pdf"`);
  res.type('application/pdf');
  orderToPdf(order, res, { company: s.company || 'FiberFleet', off: Number(s.tz_offset_min ??
180) });
});

return r;
}

// ===== Файл: server/routes/schedule.js =====
import express from 'express';
import { requirePerm } from '../auth.js';
import { json } from '../db.js';
import { fleetMaintenance } from '../domain/maintenance.js';
import { calendarFor } from '../settings.js';
import { replan } from '../services/orders.js';

const DAY = 86400000;

export default function scheduleRoutes(db) {

```

```

const r = express.Router();

const entries = (from, to, employeeId) => db.prepare(`SELECT s.id, s.task_id, s.employee_id, s
.start_at, s.end_at, s.locked,
    t.description, t.location, t.status, t.priority, t.deadline, t.hours, t.order_id, t.vehicl
e_type, t.vehicle_id, t.reason,
    o.number AS order_number, e.name AS employee_name,
    CASE t.vehicle_type WHEN 'tractor' THEN tr.plate ELSE tl.plate END AS plate,
    CASE t.vehicle_type WHEN 'tractor' THEN tr.brand || ' ' || COALESCE(tr.model, '') ELSE COAL
ESCE(tl.brand, '') || ' ' || tl.model END AS vehicle_name
FROM schedule s JOIN tasks t ON t.id = s.task_id
LEFT JOIN repair_orders o ON o.id = t.order_id
LEFT JOIN employees e ON e.id = s.employee_id
LEFT JOIN tractors tr ON t.vehicle_type = 'tractor' AND tr.id = t.vehicle_id
LEFT JOIN trailers tl ON t.vehicle_type = 'trailer' AND tl.id = t.vehicle_id
WHERE s.end_at > @from AND s.start_at < @to AND t.status <> 'cancelled' ${employeeId ? 'AND
s.employee_id = @emp' : ''}
ORDER BY s.start_at`).all({ from, to, emp: employeeId });

r.get('/schedule', (req, res) => {
    const cal = calendarFor(db);
    const from = req.query.from || cal.localDayStart(new Date());
    const to = req.query.to || new Date(new Date(from).getTime() + 7 * DAY).toISOString();
    let emp = req.query.employee_id ? Number(req.query.employee_id) : null;
    if (req.user.role === 'mechanic') emp = req.user.employee_id || -1; // слесарь видит только
своё расписание
    const employees = db.prepare('SELECT id, name, position, is_external FROM employees WHERE ac
tive = 1 ORDER BY is_external, name').all()
        .filter((e) => !emp || e.id === emp);
    res.json({ from, to, employees, entries: entries(from, to, emp) });
});

r.post('/schedule/rebuild', requirePerm('plan.write'), (_req, res) => {
    const result = replan(db);
    res.json({ placed: result.placed.size, unschedulable: result.unschedulable.size });
});

// Экран оператора: что каждый делает сейчас и что дальше
r.get('/schedule/monitor', (_req, res) => {
    const cal = calendarFor(db);
    const now = new Date();
    const dayStart = cal.localDayStart(now);
    const dayEnd = new Date(new Date(dayStart).getTime() + DAY).toISOString();
    const nowIso = now.toISOString();
    const list = entries(dayStart, new Date(now.getTime() + 3 * DAY).toISOString(), null);
    const employees = db.prepare('SELECT id, name, position, is_external FROM employees WHERE ac
tive = 1 ORDER BY is_external, name').all();
    const board = employees.map((e) => {
        const mine = list.filter((x) => x.employee_id === e.id);
        const current = mine.find((x) => x.status === 'in_progress' || x.status === 'paused')

```

```

    || mine.find((x) => x.status === 'planned' && x.start_at <= nowIso && x.end_at > nowIso)
    || null;
    const next = mine.filter((x) => x.status === 'planned' && x !== current && x.start_at >= (
current?.start_at || nowIso)).slice(0, 4);
    const today = mine.filter((x) => x.start_at < dayEnd);
    const busyMin = today.reduce((a, x) => a + Math.max(0, cal.toWork(x.end_at) - cal.toWork(x
.start_at)), 0);
    const overdue = mine.filter((x) => x.status === 'planned' && x.end_at < nowIso);
    return { ...e, current, next, today_count: today.length, load: cal.perDay ? Math.round((bu
syMin / cal.perDay) * 100) : 0, overdue: overdue.length };
    }).filter((e) => !e.is_external || e.current || e.next.length);
    res.json({ now: nowIso, board });
  });

```

```

r.get('/dashboard', (req, res) => {
  const cal = calendarFor(db);
  const now = new Date();
  const dayStart = cal.localDayStart(now);
  const dayEnd = new Date(new Date(dayStart).getTime() + DAY).toISOString();
  const orders = db.prepare(`SELECT o.id, o.number, o.status, o.release_deadline, o.plan_error
s, tr.plate AS tractor_plate, tl.plate AS trailer_plate,
    (SELECT COUNT(*) FROM tasks t WHERE t.order_id = o.id AND t.status <> 'cancelled') AS to
tal,
    (SELECT COUNT(*) FROM tasks t WHERE t.order_id = o.id AND t.status = 'done') AS done,
    (SELECT MAX(s.end_at) FROM schedule s JOIN tasks t ON t.id = s.task_id WHERE t.order_id
= o.id AND t.status <> 'cancelled') AS planned_end
  FROM repair_orders o LEFT JOIN tractors tr ON tr.id = o.tractor_id LEFT JOIN trailers tl O
N tl.id = o.trailer_id
  WHERE o.status IN ('planned','plan_error','approved','in_progress') ORDER BY o.inspected_a
t`).all()
    .map((o) => ({ ...o, errors: json(o.plan_errors, []), plan_errors: undefined }));
  const today = entries(dayStart, dayEnd, req.user.role === 'mechanic' ? req.user.employee_id
|| -1 : null);
  const maint = fleetMaintenance(db, now).filter((m) => m.status === 'overdue' || m.status ===
'soon');
  const kpi = db.prepare(`SELECT
    (SELECT COUNT(*) FROM tasks WHERE status = 'done' AND finished_at >= ?) AS done_30,
    (SELECT COALESCE(SUM(price),0) FROM tasks WHERE status = 'done' AND finished_at >= ?) AS
revenue_30,
    (SELECT COUNT(*) FROM tasks WHERE status IN ('planned','in_progress','paused')) AS open_
tasks,
    (SELECT COUNT(*) FROM tractors WHERE archived = 0) AS tractors,
    (SELECT COUNT(*) FROM trailers WHERE archived = 0) AS trailers`).get(
    new Date(now.getTime() - 30 * DAY).toISOString(), new Date(now.getTime() - 30 * DAY).toISO
String());
  res.json({ orders, today, maintenance: maint.slice(0, 20), maintenance_total: maint.length,
kpi });
});

```

```

return r;

```

```

}

// ===== Файл: server/routes/staff.js =====
import express from 'express';
import { requirePerm } from '../auth.js';
import { json, nowIso } from '../db.js';
import { eligibleEmployees, loadEmployees, speedStats } from '../domain/scheduler.js';
import { audit, replan } from '../services/orders.js';
import { bad, notFound, num } from '../util.js';

export default function staffRoutes(db) {
  const r = express.Router();

  // ----- Сотрудники -----
  r.get('/employees', (_req, res) => {
    const rows = db.prepare(`SELECT e.*,
      (SELECT COUNT(*) FROM tasks t WHERE t.employee_id = e.id AND t.status IN ('planned','in_
progress','paused')) AS open_tasks,
      (SELECT COALESCE(SUM(t.hours),0) FROM tasks t WHERE t.employee_id = e.id AND t.status IN
('planned','in_progress','paused')) AS open_hours,
      (SELECT COUNT(*) FROM work_type_access a WHERE a.employee_id = e.id) AS access_count
FROM employees e ORDER BY e.active DESC, e.is_external, e.name`).all();
    res.json(rows.map((e) => ({ ...e, specializations: json(e.specializations, []) })));
  });

  r.get('/employees/:id', (req, res) => {
    const e = db.prepare('SELECT * FROM employees WHERE id = ?').get(req.params.id);
    if (!e) throw notFound('Сотрудник');
    e.specializations = json(e.specializations, []);
    e.access = db.prepare('SELECT work_type_id FROM work_type_access WHERE employee_id = ?').all
(e.id).map((x) => x.work_type_id);
    // работы, к которым сотрудник допущен (с учётом «Все»)
    const { emps, access } = loadEmployees(db);
    const me = emps.find((x) => x.id === e.id);
    e.allowed_work_ids = me ? db.prepare('SELECT * FROM work_types WHERE active = 1').all()
      .filter((w) => eligibleEmployees(w, emps, access).some((x) => x.id === e.id)).map((w) => w
.id) : [];
    e.current_tasks = db.prepare(`SELECT t.id, t.description, t.status, t.priority, t.deadline,
t.order_id, o.number AS order_number,
      s.start_at, s.end_at, CASE t.vehicle_type WHEN 'tractor' THEN tr.plate ELSE tl.plate END
AS plate
FROM tasks t LEFT JOIN schedule s ON s.task_id = t.id LEFT JOIN repair_orders o ON o.id =
t.order_id
LEFT JOIN tractors tr ON t.vehicle_type = 'tractor' AND tr.id = t.vehicle_id
LEFT JOIN trailers tl ON t.vehicle_type = 'trailer' AND tl.id = t.vehicle_id
WHERE t.employee_id = ? AND t.status IN ('planned','in_progress','paused') ORDER BY COALES
CE(s.start_at, t.deadline)`).all(e.id);
    const stats = speedStats(db);
    e.speed_by_category = [...stats.values()].filter((s) => s.employee_id === e.id)

```

```

    .map((s) => ({ category: s.category, tasks: s.n, factor: Math.round(s.k * 100) / 100 }));
    e.totals = db.prepare(`SELECT COUNT(*) tasks, COALESCE(SUM(hours),0) norm_hours, COALESCE(SUM(
M(actual_hours),0) actual_hours,
    COALESCE(SUM(salary),0) salary FROM tasks WHERE employee_id = ? AND status = 'done'`).ge
t(e.id);
    res.json(e);
});

const empFields = (b) => {
    const out = {};
    for (const f of ['name', 'position', 'qualification', 'phone']) if (b[f] !== undefined) out[f] = (b[f] || '').trim() || null;
    if (b.level !== undefined) out.level = Math.min(3, Math.max(1, Number(b.level) || 2));
    if (b.speed_factor !== undefined) {
        const k = num(b.speed_factor);
        if (!(k >= 0.3 && k <= 3)) throw bad('Коэффициент времени – от 0,3 до 3');
        out.speed_factor = k;
    }
    if (b.specializations !== undefined) out.specializations = JSON.stringify(Array.isArray(b.sp
ecializations) ? b.specializations : []);
    for (const f of ['is_external', 'general_access', 'active']) if (b[f] !== undefined) out[f]
= b[f] ? 1 : 0;
    return out;
};

r.post('/employees', requirePerm('employees.write'), (req, res) => {
    const data = empFields(req.body || {});
    if (!data.name) throw bad('Укажите имя сотрудника');
    const cols = Object.keys(data);
    const id = db.prepare(`INSERT INTO employees(${cols.join(',')}) VALUES (${cols.map((c) => '@
' + c).join(',')})`).run(data).lastInsertRowid;
    if (Array.isArray(req.body.access)) setEmployeeAccess(db, id, req.body.access);
    audit(db, req.user, 'employee', id, 'create', data.name);
    replan(db);
    res.status(201).json({ id });
});

r.put('/employees/:id', requirePerm('employees.write'), (req, res) => {
    const e = db.prepare('SELECT id FROM employees WHERE id = ?').get(req.params.id);
    if (!e) throw notFound('Сотрудник');
    const data = empFields(req.body || {});
    if ('name' in data && !data.name) throw bad('Укажите имя сотрудника');
    if (Object.keys(data).length) {
        db.prepare(`UPDATE employees SET ${Object.keys(data).map((c) => `${c} = @${c}`).join(', ')}
WHERE id = @id`).run({ ...data, id: e.id });
    }
    if (Array.isArray(req.body.access)) setEmployeeAccess(db, e.id, req.body.access);
    if (data.active === 0) {
        // уволен/неактивен – снять с непринятых в работу задач
        db.prepare(`UPDATE tasks SET employee_id = NULL, assigned_manual = 0 WHERE employee_id = ?

```

```

    AND status = 'planned')).run(e.id);
    db.prepare(`DELETE FROM schedule WHERE employee_id = ? AND task_id IN (SELECT id FROM task
s WHERE status = 'planned')`).run(e.id);
  }
  audit(db, req.user, 'employee', e.id, 'update', JSON.stringify(req.body));
  replan(db);
  res.json({ ok: true });
});

r.delete('/employees/:id', requirePerm('employees.write'), (req, res) => {
  const id = Number(req.params.id);
  const used = db.prepare('SELECT COUNT(*) c FROM tasks WHERE employee_id = ? AND status = \'d
one\').get(id).c;
  if (used) {
    db.prepare('UPDATE employees SET active = 0 WHERE id = ?').run(id);
  } else {
    db.prepare('UPDATE tasks SET employee_id = NULL, assigned_manual = 0 WHERE employee_id = ?
').run(id);
    db.prepare('DELETE FROM employees WHERE id = ?').run(id);
  }
  audit(db, req.user, 'employee', id, used ? 'deactivate' : 'delete');
  replan(db);
  res.json({ ok: true, deactivated: !!used });
});

// ----- Справочник работ -----
r.get('/work-types', (req, res) => {
  const rows = db.prepare(`SELECT w.*, (SELECT group_concat(employee_id) FROM work_type_access
a WHERE a.work_type_id = w.id) AS access_ids,
    (SELECT COUNT(*) FROM tasks t WHERE t.work_type_id = w.id) AS used
  FROM work_types w ${req.query.all === '1' ? '' : 'WHERE w.active = 1'} ORDER BY w.category
, w.name`).all();
  res.json(rows.map((w) => ({ ...w, access: w.access_ids ? String(w.access_ids).split(',').map
(Number) : [], access_ids: undefined })));
});

const wtFields = (b) => {
  const out = {};
  for (const f of ['category', 'name', 'note', 'required_qualification']) if (b[f] !== undefin
ed) out[f] = (b[f] || '').trim() || null;
  if (b.applies_to !== undefined) out.applies_to = ['tractor', 'trailer', 'any'].includes(b.ap
plies_to) ? b.applies_to : 'any';
  for (const f of ['hours', 'hours_max', 'price', 'price_max', 'salary', 'salary_max']) if (b[
f] !== undefined) out[f] = num(b[f]);
  for (const f of ['access_all', 'external', 'active']) if (b[f] !== undefined) out[f] = b[f]
? 1 : 0;
  if ('hours' in out && !(out.hours > 0)) throw bad('Норма времени должна быть больше 0');
  return out;
};

```

```

r.post('/work-types', requirePerm('catalog.write'), (req, res) => {
  const data = wtFields(req.body || {});
  if (!data.name || !data.category) throw bad('Укажите категорию и название работы');
  if (!data.hours) throw bad('Укажите норму времени');
  const last = db.prepare("SELECT code FROM work_types WHERE code LIKE 'W%' ORDER BY CAST(SUBSTR(code, 2) AS INTEGER) DESC LIMIT 1").get();
  data.code = req.body.code?.trim() || `W${String((last ? Number(last.code.slice(1)) : 0) + 1).padStart(3, '0')}`;
  const cols = Object.keys(data);
  const id = db.prepare(`INSERT INTO work_types(${cols.join(',')}) VALUES (${cols.map((c) => '@' + c).join(',')})`).run(data).lastInsertRowid;
  if (Array.isArray(req.body.access)) setWorkAccess(db, id, req.body.access);
  audit(db, req.user, 'work_type', id, 'create', data.name);
  res.status(201).json({ id });
});

```

```

r.put('/work-types/:id', requirePerm('catalog.write'), (req, res) => {
  const w = db.prepare('SELECT * FROM work_types WHERE id = ?').get(req.params.id);
  if (!w) throw notFound('Работа');
  const data = wtFields(req.body || {});
  if (req.body.code !== undefined && req.body.code.trim()) data.code = req.body.code.trim();
  if (Object.keys(data).length) {
    db.prepare(`UPDATE work_types SET ${Object.keys(data).map((c) => `${c} = @${c}`).join(',')}`, updated_at = @ts WHERE id = @id`)
      .run({ ...data, ts: nowIso(), id: w.id });
  }
  if (Array.isArray(req.body.access)) setWorkAccess(db, w.id, req.body.access);
  // обновить цену/норму в ещё не начатых работах
  if (req.body.apply_to_open) {
    db.prepare(`UPDATE tasks SET hours = @hours, price = @price, salary = @salary WHERE work_type_id = @id AND status = 'planned'`)
      .run({ id: w.id, hours: data.hours ?? w.hours, price: data.price ?? w.price, salary: data.salary ?? w.salary });
  }
  audit(db, req.user, 'work_type', w.id, 'update', JSON.stringify(req.body));
  if (req.body.access || 'access_all' in data || 'external' in data || req.body.apply_to_open) replan(db);
  res.json({ ok: true });
});

```

```

r.delete('/work-types/:id', requirePerm('catalog.write'), (req, res) => {
  const id = Number(req.params.id);
  const used = db.prepare('SELECT COUNT(*) c FROM tasks WHERE work_type_id = ?').get(id).c;
  if (used) db.prepare('UPDATE work_types SET active = 0 WHERE id = ?').run(id);
  else db.prepare('DELETE FROM work_types WHERE id = ?').run(id);
  audit(db, req.user, 'work_type', id, used ? 'deactivate' : 'delete');
  res.json({ ok: true, deactivated: !!used });
});

```

```

return r;

```

```

}

function setEmployeeAccess(db, employeeId, workTypeIds) {
  db.transaction(() => {
    db.prepare('DELETE FROM work_type_access WHERE employee_id = ?').run(employeeId);
    const ins = db.prepare('INSERT OR IGNORE INTO work_type_access(work_type_id, employee_id) VALUES (?, ?)');
    for (const w of workTypeIds) ins.run(Number(w), employeeId);
  })();
}

function setWorkAccess(db, workTypeId, employeeIds) {
  db.transaction(() => {
    db.prepare('DELETE FROM work_type_access WHERE work_type_id = ?').run(workTypeId);
    const ins = db.prepare('INSERT OR IGNORE INTO work_type_access(work_type_id, employee_id) VALUES (?, ?)');
    for (const e of employeeIds) ins.run(workTypeId, Number(e));
  })();
}

// ===== Файл: server/routes/vehicles.js =====
import express from 'express';
import { requirePerm } from '../auth.js';
import { nowIso } from '../db.js';
import { recordMileage, vehicleMaintenance } from '../domain/maintenance.js';
import { audit } from '../services/orders.js';
import { bad, normalizePlate, notFound, num } from '../util.js';

const TABLE = { tractor: 'tractors', trailer: 'trailers' };
const FIELDS = {
  tractor: ['plate', 'brand', 'model', 'year', 'vin', 'mileage', 'axles', 'brake_type', 'trailer_id', 'notes', 'archived'],
  trailer: ['plate', 'brand', 'model', 'year', 'vin', 'mileage', 'axles', 'brake_type', 'has_reefer', 'reefer_hours', 'notes', 'archived'],
};

function clean(type, body) {
  const out = {};
  for (const f of FIELDS[type]) {
    if (body[f] === undefined) continue;
    let v = body[f];
    if (['year', 'mileage', 'axles', 'trailer_id', 'reefer_hours'].includes(f)) v = num(v);
    if (['archived', 'has_reefer'].includes(f)) v = v ? 1 : 0;
    if (f === 'plate') v = normalizePlate(v);
    if (typeof v === 'string') v = v.trim() || null;
    out[f] = v;
  }
  if ('plate' in out && !out.plate) throw bad('Укажите госномер');
  if ('axles' in out && (out.axles < 1 || out.axles > 5)) throw bad('Количество осей: от 1 до 5');
}

```

```

);
if ('brake_type' in out && !['disc', 'drum'].includes(out.brake_type)) out.brake_type = 'disc'
;
return out;
}

```

```

export default function vehicleRoutes(db) {
  const r = express.Router();

  r.get('/vehicles/:type(tractor|trailer)', (req, res) => {
    const { type } = req.params;
    const q = `%${normalizePlate(req.query.q || '')}%`;
    const qText = `%${String(req.query.q || '').toUpperCase()}%`;
    const archived = req.query.archived === '1' ? 1 : 0;
    const rows = type === 'tractor'
      ? db.prepare(`SELECT v.*, t.plate AS trailer_plate, t.model AS trailer_model,
        (SELECT COUNT(*) FROM repair_orders o WHERE o.tractor_id = v.id AND o.status IN ('planned', 'plan_error', 'approved', 'in_progress')) AS open_orders
        FROM tractors v LEFT JOIN trailers t ON t.id = v.trailer_id
        WHERE v.archived = ? AND (v.plate LIKE ? OR UPPER(v.brand || ' ' || COALESCE(v.model, '
        ')) LIKE ?)
        ORDER BY v.brand, v.plate`).all(archived, q, qText)
      : db.prepare(`SELECT v.*, t.id AS tractor_id, t.plate AS tractor_plate,
        (SELECT COUNT(*) FROM repair_orders o WHERE o.trailer_id = v.id AND o.status IN ('planned', 'plan_error', 'approved', 'in_progress')) AS open_orders
        FROM trailers v LEFT JOIN tractors t ON t.trailer_id = v.id
        WHERE v.archived = ? AND (v.plate LIKE ? OR UPPER(COALESCE(v.brand, ' ') || ' ' || v.model) LIKE ?)
        ORDER BY v.brand, v.plate`).all(archived, q, qText);
    res.json(rows);
  });

  r.get('/vehicles/:type(tractor|trailer)/:id', (req, res) => {
    const { type, id } = req.params;
    const v = db.prepare(`SELECT * FROM ${TABLE[type]} WHERE id = ?`).get(id);
    if (!v) throw notFound('TC');
    if (type === 'tractor' && v.trailer_id) v.trailer = db.prepare('SELECT id, plate, model FROM trailers WHERE id = ?').get(v.trailer_id);
    if (type === 'trailer') v.tractor = db.prepare('SELECT id, plate, brand, model FROM tractors WHERE trailer_id = ?').get(v.id) || null;
    const col = type === 'tractor' ? 'tractor_id' : 'trailer_id';
    v.orders = db.prepare(`SELECT id, number, status, inspected_at, release_deadline, closed_at,
      ${type === 'tractor' ? 'tractor_mileage' : 'trailer_mileage'} AS mileage,
      (SELECT COUNT(*) FROM tasks t WHERE t.order_id = o.id) AS tasks_total,
      (SELECT COUNT(*) FROM tasks t WHERE t.order_id = o.id AND t.status = 'done') AS tasks_done
      FROM repair_orders o WHERE ${col} = ? ORDER BY inspected_at DESC`).all(id);
    v.history = db.prepare(`SELECT t.id, t.order_id, o.number AS order_number, t.description, t.location, t.status, t.registered_at,
      t.finished_at, t.mileage, t.price, t.hours, t.actual_hours, t.reason, e.name AS employee

```

```

_name, w.category
  FROM tasks t LEFT JOIN repair_orders o ON o.id = t.order_id LEFT JOIN employees e ON e.id
= t.employee_id
  LEFT JOIN work_types w ON w.id = t.work_type_id
  WHERE t.vehicle_type = ? AND t.vehicle_id = ? ORDER BY COALESCE(t.finished_at, t.registere
d_at) DESC`).all(type, id);
  v.mileage_log = db.prepare(`SELECT * FROM mileage_log WHERE vehicle_type = ? AND vehicle_id
= ? ORDER BY recorded_at DESC LIMIT 100`).all(type, id);
  v.parts = db.prepare(`SELECT * FROM installed_parts WHERE vehicle_type = ? AND vehicle_id =
? ORDER BY installed_at DESC`).all(type, id);
  v.maintenance = vehicleMaintenance(db, type, v);
  v.totals = db.prepare(`SELECT COUNT(*) works, COALESCE(SUM(price),0) price FROM tasks WHERE
vehicle_type = ? AND vehicle_id = ? AND status = 'done'`).get(type, id);
  res.json(v);
});

r.post('/vehicles/:type(tractor|trailer)', requirePerm('vehicles.write'), (req, res) => {
  const { type } = req.params;
  const data = clean(type, req.body || {});
  if (!data.plate) throw bad('Укажите госномер');
  if (type === 'tractor' && !data.brand) throw bad('Укажите марку');
  if (type === 'trailer' && !data.model) throw bad('Укажите модель');
  const cols = Object.keys(data);
  const id = db.prepare(`INSERT INTO ${TABLE[type]}(${cols.join(',')}) VALUES (${cols.map((c)
=> '@' + c).join(',')})`).run(data).lastInsertRowid;
  if (data.mileage) recordMileage(db, { type, id, mileage: data.mileage, source: 'manual', use
rId: req.user.id });
  if (type === 'tractor' && data.trailer_id) db.prepare('UPDATE tractors SET trailer_id = NULL
WHERE trailer_id = ? AND id <> ?').run(data.trailer_id, id);
  audit(db, req.user, type, id, 'create', data.plate);
  res.status(201).json({ id });
});

r.put('/vehicles/:type(tractor|trailer)/:id', requirePerm('vehicles.write'), (req, res) => {
  const { type, id } = req.params;
  const cur = db.prepare(`SELECT * FROM ${TABLE[type]} WHERE id = ?`).get(id);
  if (!cur) throw notFound('TC');
  const data = clean(type, req.body || {});
  const mileage = data.mileage;
  delete data.mileage; // пробег – только через журнал
  if (Object.keys(data).length) {
    db.prepare(`UPDATE ${TABLE[type]} SET ${Object.keys(data).map((c) => `${c} = @${c}`).join(
', ')} , updated_at = @ts WHERE id = @id`)
      .run({ ...data, ts: nowIso(), id });
  }
  if (mileage != null && mileage !== cur.mileage) {
    recordMileage(db, { type, id: Number(id), mileage, source: 'manual', userId: req.user.id,
allowDecrease: req.body.allow_decrease === true });
  }
  if (type === 'tractor' && data.trailer_id) db.prepare('UPDATE tractors SET trailer_id = NULL

```

```

WHERE trailer_id = ? AND id <> ?').run(data.trailer_id, id);
// сцепка со стороны прицепа
if (type === 'trailer' && req.body.tractor_id !== undefined) {
  db.prepare('UPDATE tractors SET trailer_id = NULL WHERE trailer_id = ?').run(id);
  if (req.body.tractor_id) db.prepare('UPDATE tractors SET trailer_id = ? WHERE id = ?').run
(id, req.body.tractor_id);
}
audit(db, req.user, type, Number(id), 'update', JSON.stringify(req.body));
res.json({ ok: true });
});

```

```

r.delete('/vehicles/:type(tractor|trailer)/:id', requirePerm('vehicles.write'), (req, res) =>
{
  const { type, id } = req.params;
  const used = db.prepare('SELECT COUNT(*) c FROM tasks WHERE vehicle_type = ? AND vehicle_id
= ?').get(type, id).c
  + db.prepare(`SELECT COUNT(*) c FROM repair_orders WHERE ${type} === 'tractor' ? 'tractor_i
d' : 'trailer_id' = ?`).get(id).c;
  if (used && req.query.hard !== '1') {
    // есть история ремонтов – переносим в архив, чтобы не потерять историю
    db.prepare(`UPDATE ${TABLE[type]} SET archived = 1, updated_at = ? WHERE id = ?`).run(nowI
so(), id);
    audit(db, req.user, type, Number(id), 'archive');
    return res.json({ ok: true, archived: true });
  }
  if (type === 'trailer') db.prepare('UPDATE tractors SET trailer_id = NULL WHERE trailer_id =
?').run(id);
  db.prepare('DELETE FROM mileage_log WHERE vehicle_type = ? AND vehicle_id = ?').run(type, id
);
  db.prepare(`DELETE FROM ${TABLE[type]} WHERE id = ?`).run(id);
  audit(db, req.user, type, Number(id), 'delete');
  res.json({ ok: true, deleted: true });
});

```

```

r.post('/vehicles/:type(tractor|trailer)/:id/mileage', requirePerm('vehicles.write'), (req, re
s) => {
  const { type, id } = req.params;
  const mileage = num(req.body?.mileage);
  if (mileage == null || mileage < 0) throw bad('Укажите пробег');
  const cur = db.prepare(`SELECT mileage FROM ${TABLE[type]} WHERE id = ?`).get(id);
  if (!cur) throw notFound('TC');
  if (mileage < cur.mileage && !req.body.allow_decrease) {
    throw bad(`Пробег ${mileage} км меньше текущего (${cur.mileage} км). Если это исправление
ошибки – подтвердите уменьшение`);
  }
  recordMileage(db, { type, id: Number(id), mileage, at: req.body.at || nowIso(), source: 'man
ual', userId: req.user.id, allowDecrease: !!req.body.allow_decrease });
  res.json({ ok: true });
});

```

```
    return r;  
}
```